

Memory in the LLM Era: Modular Architectures and Strategies in a Unified Framework [Experiment, Analysis & Benchmark]

Yanchen Wu*
CUHK-Shenzhen

Tenghui Lin*
CUHK

Yingli Zhou†
CUHK-Shenzhen

Fangyuan Zhang†
HITSZ

Qintian Guo
BIT

Xun Zhou
HITSZ

Sibo Wang
CUHK

Xilin Liu
Huawei Cloud

Yuchi Ma
Huawei Cloud

Yixiang Fang
CUHK-Shenzhen

ABSTRACT

Memory emerges as the core module in the large language model (LLM)-based agents for long-horizon complex tasks (e.g., multi-turn dialogue, game playing, scientific discovery), where memory can enable knowledge accumulation, iterative reasoning and self-evolution. A number of memory methods have been proposed in the literature. However, these methods have not been systematically and comprehensively compared under the same experimental settings. In this paper, we first summarize a unified framework that covers existing representative agent memory methods from a high-level perspective. We then extensively compare representative agent memory methods on two long-term conversational benchmarks and an agentic memory benchmark, and examine the effectiveness of representative methods, providing a thorough analysis of those methods. As a byproduct of our experimental analysis, we also design a new memory method by exploiting modules in the existing methods, which outperforms the state-of-the-art methods. Finally, based on these findings, we offer promising future research opportunities. We believe that a deeper understanding of the behavior of existing methods can provide valuable new insights for future research.

PVLDB Reference Format:

Yanchen Wu, Tenghui Lin, Yingli Zhou, Fangyuan Zhang, Qintian Guo, Xun Zhou, Sibow Wang, Xilin Liu, Yuchi Ma, and Yixiang Fang. Memory in the LLM Era: Modular Architectures and Strategies in a Unified Framework [Experiment, Analysis & Benchmark]. PVLDB, 19(1): XXX-XXX, 2026. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Yanchen398/Memory-in-the-LLM-Era>.

1 INTRODUCTION

The development of Large Language Models (LLMs) like GPT-5 [95], Qwen3 [119], and Claude Sonnet 4.6 [3] has sparked a revolution in the field of artificial intelligence [22, 36, 57, 65, 78, 104, 106, 127]. Building on this success, LLM-powered agents have rapidly emerged and are being deployed across a wide range of domains,

*The first two authors contributed equally to this research.

†These authors jointly led this project.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.

doi:XX.XX/XXX.XX

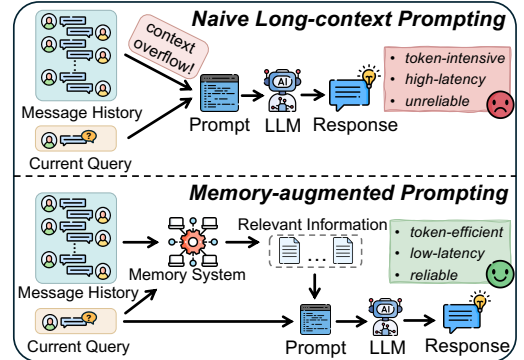


Figure 1: Overview of naive long-context prompting and memory-augmented prompting.

from industrial automation to personal assistance. For example, SWE-agent systems for software engineering tasks [121] and personal assistant agents such as OpenClaw [79] illustrate how LLM-based agents can autonomously plan, reason, and execute complex multi-step workflows. These agents are increasingly expected to operate autonomously, adapt to diverse environments, and support personalized interactions tailored to user needs. A key capability underlying such intelligent behavior is the *memory mechanism* [84, 124]. As illustrated in Figure 1, memory mechanisms allow agents to move beyond naive long-context prompting by maintaining and leveraging relevant information from past interactions. By equipping agents with memory mechanisms, they can accumulate experience over time, maintain contextual knowledge, and make more informed decisions—analogueous to how humans rely on memory to learn from past experiences and guide future actions.

In recent years, a growing number of memory methods have been proposed to enhance the capability of LLM-based agents to retain, organize, and utilize historical information across interactions. These methods aim to enable agents to move beyond stateless reasoning and instead support long-term planning, personalization, and adaptive decision-making. Motivated by the limitations of stateless LLM agents and the growing need for persistent contextual reasoning, researchers from multiple communities—including databases, data mining, machine learning, and natural language processing—have begun developing efficient and scalable memory mechanisms for intelligent agents [42, 81, 82, 84, 89, 105, 107, 109, 110, 132]. In Table 1, we summarize twelve representative agent memory methods. We categorize them according to four key dimensions: information extraction mechanism, memory management strategy, underlying storage structure, and retrieval method. After a careful literature review, we make the following observations.

Table 1: Classification of representative agent memory methods.

Method	Information Extraction	Management Operations	Storage Structure	Retrieval Mechanism
A-MEM [115]	Direct archive, Summarization-based extract	Connect, Update	Flat, Vector	Vector-Based
MemoryBank [128]	Direct archive	Integrate, Update, Filter	Flat, Vector	Vector-Based
MemGPT [82]	Direct archive	Integrate, Transform, Update	Hierarchical, Vector	Lexical-Based, Vector-Based
Mem0 [12]	Direct archive, Summarization-based extract	Integrate, Update, Filter	Flat, Vector	Vector-Based
Mem0 ^g [12]	Graph-based extract	Connect, Update, Filter	Flat, Graph	Vector-Based, Structure-Based
MemoChat [70]	Direct archive	Integrate	Flat	LLM-Assisted
Zep [88]	Direct archive, Graph-based extract	Connect, Transform, Update	Hierarchical, Graph	Lexical-Based, Vector-Based, Structure-Based
MemTree [89]	Direct archive	Connect, Integrate, Update	Flat, Tree	Vector-Based
MemoryOS [43]	Direct archive	Connect, Integrate, Transform, Update, Filter	Hierarchical, Vector	Lexical-Based, Vector-Based
MemOS [58]	Direct archive, Summarization-based extract	Connect, Integrate, Update	Hierarchical, Tree	Lexical-Based, Vector-Based
MemGAS [116]	Direct archive, Summarization-based extract	Connect	Flat, Graph	Vector-Based, Structure-Based, LLM-Assisted
LightMem [18]	Direct archive, Summarization-based extract	Integrate, Update, Filter	Hierarchical, Vector	Vector-Based

First, there is a lack of a unified framework for systematically analyzing the implementation components of representative agent memory methods. Second, most previous studies report overall performance results, but rarely examine the roles and effects of individual components within these methods. Third, comprehensive and systematic comparisons among different methods—especially regarding their accuracy and efficiency—are still lacking.

Our work. We address these gaps by proposing a unified, modular framework and conducting an in-depth experimental study of representative agent memory methods. The framework decomposes memory mechanisms into four stages, including ❶ *Information Extraction*, ❷ *Memory Management*, ❸ *Memory Storage*, and ❹ *Information Retrieval*. Under this framework, we compare representative methods on two typical long-term conversational benchmarks, **LOCOMO** [72] and **LONGMEMEVAL** [111], as well as the agentic memory benchmark **MemoryArena** [29]. Beyond overall performance, we analyze practical robustness dimensions, including context scalability and positional sensitivity. Based on these analyses, we further design a new agent memory method that achieves the best overall performance and cost efficiency.

In summary, our principal contributions are listed as follows:

- We propose a unified framework that decomposes representative agent memory methods into four core memory modules, enabling systematic comparisons of their differences.
- We conduct comprehensive experimental studies across conversational and agentic settings using LOCOMO, LONGMEMEVAL, and MemoryArena, together with analyses of efficiency, context scalability, evidence position sensitivity, and LLM backbone dependence.
- Based on the above analyses, we propose a new agent memory method that achieves state-of-the-art performance among the evaluated representative baselines. We further derive several key insights and highlight promising research directions for future work.

Roadmap. Section 2 introduces preliminaries. Section 3 presents the unified framework. Sections 4–7 characterize representative design choices within this framework. Section 8 reports experimental results and analyses. Section 9 summarizes lessons and opportunities. Section 10 reviews related work, and Section 11 concludes.

2 PRELIMINARIES

In this section, we go through some important concepts and the typical workflow of existing memory methods in the LLM era. The relationship between RAG and memory is also discussed.

2.1 LLM-related Concepts

We introduce two fundamental LLM-related concepts below.

LLM Prompting. Prompting [6, 13, 68] specifies an LLM task by constructing an input context (prompt) that contains task instructions, the current input, and optionally demonstrations or auxiliary evidence. The model’s output is then generated conditioned on this context.

Prompting is particularly important in LLM-based systems because it provides a lightweight, training-free interface for task adaptation [68]. Model behavior can be redirected by revising the input context without modifying model parameters. This property has enabled a wide range of practical applications in which task specifications and constraints are expressed directly in natural language [1, 80]. In addition to final response generation, prompting is often used to drive intermediate subtasks in an LLM-centered pipeline [4, 45, 92, 122]. Typical examples include extracting key information, consolidating intermediate results, and so on.

LLM-based Agents. An LLM-based agent utilizes an LLM as a core decision model for sequential action selection [54, 94, 122]. In contrast to single-turn prompting, an agent operates in a closed loop: it receives observations, then performs reasoning or planning, executes an action (possibly via tools), obtains feedback from the environment, and further proceeds to the next step [120]. The interaction history and available state are combined into textual context, based on which the agent predicts the next action.

The action space of an LLM-based agent typically includes both natural-language responses and structured tool calls [58, 82] (e.g., information search, API invocation and memory read/write operations), which enable multi-step task completion beyond one-shot generation. Therefore, the LLM-based agent must retain and reuse information across conversation turns and sessions, which motivates effective memory mechanisms.

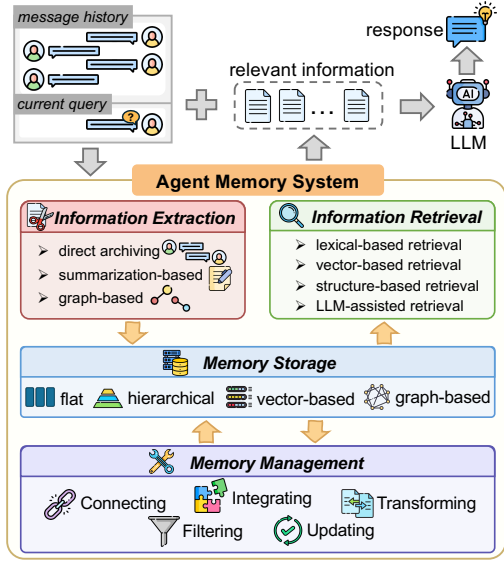


Figure 2: An overview of the unified framework for agent memory systems.

2.2 Agent Memory

Memory is incorporated into LLM-based agents to compensate for the bounded context window [31, 66, 72]. Since the model conditions only on a limited number of tokens, information that comes from earlier turns and lies outside the current prompt can be easily lost, which degrades performance in long-horizon dialogue and multi-session tasks [93, 111, 117]. An explicit memory module [5, 24, 82, 84, 99] allows the system to persist interaction-derived information—such as user preferences, salient events, intermediate decisions, and task constraints—and to reintroduce it when relevant, thereby improving consistency and enabling reasoning that depends on long-term context [24, 84].

The typical workflow of memory-augmented systems begins by selectively extracting important information from ongoing interactions—such as facts, user preferences, or important events—and storing them as memory entries. These entries then undergo a series of management operations: consolidation [15, 103, 128], which integrates similar memories to improve coherence and reduce redundancy; updating [32, 67, 82], which modifies stored content to maintain accuracy and reflect the latest knowledge; filtering [55, 84, 128], which removes outdated, redundant, or low-utility memories to preserve the efficiency and relevance of the system; and enhancement [33, 100], which marks or surfaces important memories for easy identification and retrieval. This structured process ensures that the memory system remains organized, scalable, and aligned with ongoing user needs. When additional context is required for reasoning or generation, the memory system retrieves and supplies the most relevant information to the LLM, supporting both short-term adaptation and long-term continuity across evolving interactions.

Memory and retrieval-augmented generation (RAG) [21, 27, 48, 105] are related but distinct mechanisms. Memory primarily targets stateful, interaction-dependent information that evolves over

time and is required for personalization and cross-session continuity [82, 128]. In contrast, RAG primarily targets external knowledge grounding, retrieving evidence from document collections or knowledge bases to supplement domain knowledge and reduce hallucinations [21, 48]. In practice, they are complementary: memory supplies user- and session-specific context, while RAG provides task-relevant factual evidence from external corpora.

3 A UNIFIED FRAMEWORK

In this section, we decompose existing agent memory systems into memory modules under a unified framework, as illustrated in Figure 2. The framework comprises four core memory modules: ① *Information extraction*, ② *Memory management*, ③ *Memory storage*, and ④ *Information retrieval*. Taken together, these modules capture how existing agent memory systems operate in practice.

As shown in Algorithm 1, at interaction step t , let $\mathcal{M}_t$ denote the incoming messages, q_t the current query, $\mathcal{D}_{t-1}$ the existing logical memory records, and $\mathcal{H}_{t-1}$ the materialized memory state. The four modules are formalized as

$$\begin{aligned} \mathcal{E}_t &= \text{Extract}_{\theta_E}(\mathcal{M}_t), & \Delta_t &= \text{Manage}_{\theta_M}(\mathcal{H}_{t-1}, \mathcal{E}_t), \\ \mathcal{H}_t &= \text{Materialize}_{\theta_S}(\mathcal{H}_{t-1}, \Delta_t), & \mathcal{X}_t &= \text{Retrieve}_{\theta_R}(q_t, \mathcal{H}_t). \end{aligned}$$

Specifically, ① *information extraction* performs data ingestion and transformation, converting raw messages and their associated metadata, such as timestamps, into logical memory records $\mathcal{E}_t$. It filters out redundant details and transforms the relevant content into different types of knowledge (e.g., triples derived from text, informational summaries) suitable for downstream processing. ② *Memory management* performs incremental maintenance over the existing memory state $\mathcal{H}_{t-1}$. Given the extracted records $\mathcal{E}_t$, it produces a change set Δ_t that may connect related records, integrate fragmented information, migrate records across memory levels, update existing content, or remove redundant and obsolete entries. ③ *Memory storage* materializes Δ_t into the updated persistent state $\mathcal{H}_t = \langle \mathcal{D}_t, \mathcal{P}_t, \mathcal{I}_t \rangle$. Here, $\mathcal{D}_t$ contains the logical memory records and their associated metadata, $\mathcal{P}_t$ specifies their flat or hierarchical organization, and $\mathcal{I}_t$ maintains vector- or graph-based representations together with their associated indexes and access structures. ④ *Information retrieval* performs query processing over the materialized memory state $\mathcal{H}_t$. Given the current query q_t , it identifies candidate records, accesses them through the corresponding indexes or structures, and ranks their relevance to produce the retrieved context $\mathcal{X}_t$ for downstream reasoning or response generation.

4 INFORMATION EXTRACTION

This module serves to identify and extract information from $\mathcal{M}_t$ that is both useful and necessary for downstream memory processing. As illustrated in Figure 3, existing agent systems adopt different information extraction methods, which can be broadly categorized as follows:

- ① **Direct archiving.** This method represents the most straightforward form of information extraction, where the agent system simply archives raw messages and timestamps without any processing.
- ② **Summarization-based extraction.** This method employs LLMs to generate concise informational summaries from one or more dialogue turns. Memory methods such as A-MEM and Mem0 extract

Algorithm 1: Data management workflow of the unified agent memory framework

Input : Incoming messages M_t , query q_t , persistent memory state $\mathcal{H}_{t-1} = \langle \mathcal{D}_{t-1}, \mathcal{P}_{t-1}, \mathcal{I}_{t-1} \rangle$, and module configurations Θ

Output : Updated memory state $\mathcal{H}_t$, retrieved context $\mathcal{X}_t$, and response $\mathcal{R}_t$

```

// ① Information extraction
1  $\mathcal{E}_t \leftarrow \text{Extract}_{\theta_E}(M_t);$  // construct logical memory records and metadata
// ② Memory management
2  $\Delta_t \leftarrow \text{Manage}_{\theta_M}(\mathcal{H}_{t-1}, \mathcal{E}_t);$  // connect, integrate, transform, update, or filter records
// ③ Memory storage
3  $\mathcal{H}_t \leftarrow \text{Materialize}_{\theta_S}(\mathcal{H}_{t-1}, \Delta_t);$  // persist records and maintain organization and indexes
// ④ Information retrieval
4  $\mathcal{X}_t \leftarrow \text{Retrieve}_{\theta_R}(q_t, \mathcal{H}_t);$  // retrieve and rank relevant memory records
5  $\mathcal{R}_t \leftarrow \text{GenerateResponse}(q_t, \mathcal{X}_t);$ 
6 return  $(\mathcal{H}_t, \mathcal{X}_t, \mathcal{R}_t)$ 

```

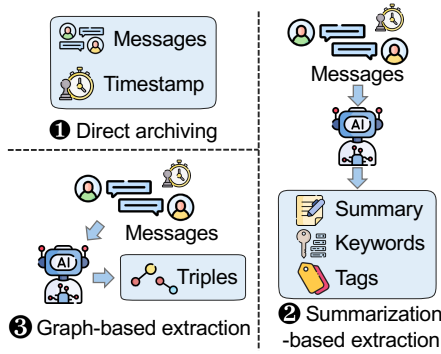


Figure 3: Methods of information extraction.

keywords and contextual tags from M_t , or prompt the LLM to produce an abstracted summary of the raw text. Figure 4 illustrates a representative prompt used for this extraction method.

③ Graph-based extraction. This method leverages LLMs to extract fine-grained entities and relations from M_t , forming subject–predicate–object triples for knowledge graph construction (e.g., Mem0^g, Zep). Additionally, temporal metadata such as creation or invalidation timestamps are recorded to support dynamic updates and temporal reasoning within the graph-based memory. A concrete example of the prompt designed for graph-based extraction is provided in Appendix A.

5 MEMORY MANAGEMENT

Memory management incrementally maintains and refines the existing logical memory records $\mathcal{D}_{t-1}$ using newly extracted records $\mathcal{E}_t$, producing a change set Δ_t that governs how memory evolves over time. As illustrated in Figure 5, it mirrors the human memory lifecycle, encompassing five core operations: connecting related experiences, integrating fragmented information, transforming short-term into long-term memory, updating outdated content, and filtering obsolete knowledge. Table 2 summarizes how representative

Prompt for Summarization-based Extraction.

Generate a structured analysis of the following message by extracting salient keywords, identifying core themes and contextual elements, and assigning relevant categorical tags.

Format the response as a JSON object:

```

{
  "summary": "<one-sentence gist>",
  "keywords": [
    <several keywords capturing key concepts>
  ],
  "tags": [
    <several broad themes for classification>
  ]
}

```

Message for analysis:
{message}

Figure 4: A sample prompt for summarization-based extraction.

agent memory methods instantiate these operations through different implementation paradigms. Through this process, the system maintains a coherent, efficient, and adaptive memory state that supports continual learning and reasoning.

① Connecting Related Experiences. Humans naturally associate related events across time and context; agent systems emulate this through *connection*. This mechanism establishes explicit connections between memory entries that share semantic similarity, temporal proximity, or contextual relevance, realized through either **structural edges** within a graph or **associative links** across discrete records. For example, memory methods such as A-MEM and MemoryOS leverage associative links based on semantic similarity or continuity, enabling synchronous updates and alignment across connected memories. Separately, graph-based methods such as Zep and Mem0^g focus on connecting individual episode or entity nodes to support reasoning and retrieval across conceptually or temporally aligned memories.

② Integrating Fragmented Memories. Humans tend to summarize daily experiences, retaining only key events while discarding details. Agent systems achieve similar *integration* through **abstraction** or **summarization**. For example, MemoryBank aggregates repetitive daily records into event summaries and refines a global user profile as experiences accumulate. This process reduces redundancy, distills essential information, and transforms scattered memories into concise high-level representations suitable for long-term storage.

③ Transforming Across Memory Levels. Human memory gradually transfers important information from low-level to high-level storage, reinforcing what is repeatedly recalled. Agent systems adopt similar hierarchical migration mechanisms. For instance, MemoryOS implements a two-stage migration strategy: short-term memories are first moved to mid-term storage following a First-In, First-Out (FIFO) policy, and mid-term memories are then promoted to long-term storage using a heat-based score that jointly considers access frequency and recency. Additionally, memory methods

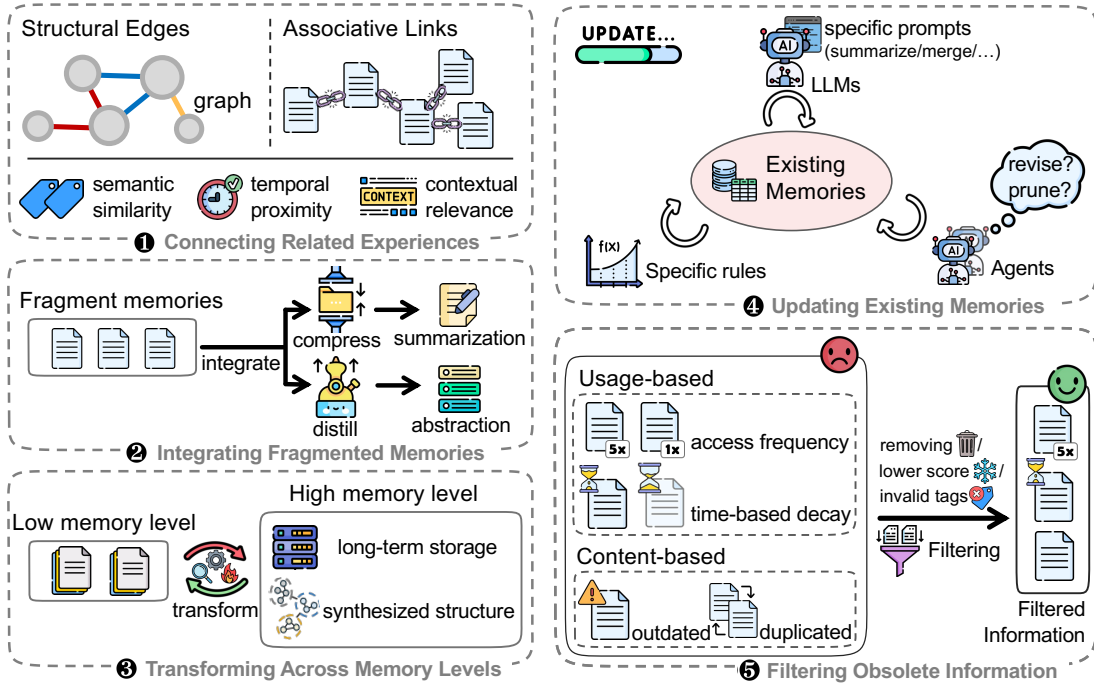


Figure 5: Workflow of the memory management process.

such as Zep organize semantically related memories into communities, forming structured, interconnected long-term representations. This stage strengthens persistent knowledge while maintaining efficiency.

④ Updating Existing Memories. Humans constantly revise their memories by integrating new experiences and correcting inconsistencies. Agent systems follow a similar principle through three main updating paradigms: (1) **Rule-based updating**, where existing memories are updated according to predefined rules. For example, MemoryBank adopts Ebbinghaus’s Forgetting Curve theory to adjust memory strength over time. In MemoryOS, new memories are integrated into existing structures based on semantic and keyword similarities; (2) **LLM-based updating**, where large language models are prompted to summarize, merge, or resolve conflicts between entries. To give an example, MemTree updates its memory by relying on the LLM to execute a specialized Aggregate Operation, where the prompt and the count of descendants guide the LLM to appropriately compress and generalize information before the new content is written back to the parent node. Taking another example, the update process in Zep requires the LLM to execute resolution tasks by strictly following detailed semantic constraints and procedural guidelines given in the prompts. (3) **Agent-based updating**, where agents autonomously decide which operations (e.g., revise, merge, prune) to apply, as in MemGPT and MemOS. To be more specific, agents are granted access to current context as well as historical or archival memory entries and learn to utilize specialized system tools for managing memories efficiently and flexibly. These strategies ensure that memory remains accurate, consistent, and aligned with evolving knowledge.

⑤ Filtering Obsolete Information. Finally, the memory system must remain compact and relevant by filtering outdated or

redundant information, which can be achieved by either directly removing the memories, lowering their assigned weight score, or applying a status label (such as “invalid”). This *filtering* process parallels human forgetting, which selectively fades unused or irrelevant memories. (1) **Usage-based filtering**, as seen in MemoryOS and MemoryBank, relies on access frequency and time-based decay. Memories created a long time ago and rarely retrieved are filtered first. (2) **Content-based filtering** examines semantic similarity and leverages LLMs to detect and filter duplicated or outdated knowledge, such as in Mem0 and Mem0^g, reducing noise and improving retrieval precision. Together, these mechanisms sustain an efficient, lightweight memory that supports ongoing adaptation and learning.

6 MEMORY STORAGE

Memory storage materializes the change set Δ_t into the updated persistent memory state $\mathcal{H}_t$, governing how logical memory records $\mathcal{D}_t$ are organized and represented along two primary dimensions: organization-centric and representation-centric. The former determines the architectural depth of the storage system, encompassing flat storage and hierarchical storage, and the latter characterizes the employed technological paradigm, primarily comprising vector-based storage and graph-based storage.

① Flat storage. This approach represents a unified, single-tier storage that aggregates all information within a homogeneous space, such as a FIFO queue or a JSON file, defined relative to hierarchical storage within the organization-centric dimension.

② Hierarchical storage. This approach partitions memory into specialized, multi-tiered architectures, allowing individual storage components to fulfill distinct functional roles and operate at different levels of granularity. For example, MemoryOS organizes memory

Table 2: Comparison of implementation paradigms for memory management operations; “N/A” means that this operation is not explicitly implemented.

Method	Connecting	Integrating	Transforming	Updating	Filtering
A-MEM	Associative Links	N/A	N/A	LLM-based	N/A
MemoryBank	N/A	Summarization	N/A	Rule-based	Usage-based
MemGPT	N/A	Summarization	Stage-wise Transfer	Agent-based	N/A
Mem0	N/A	Summarization	N/A	Agent-based	Content-based
Mem0 ^g	Structural Edges	N/A	N/A	LLM-based	Content-based
MemoChat	N/A	Abstraction	N/A	N/A	N/A
Zep	Structural Edges	N/A	Community Formation	LLM-based	N/A
MemTree	Structural Edges	Summarization	N/A	LLM-based	N/A
MemoryOS	Associative Links	Abstraction + Summarization	Stage-wise Transfer	Rule-based	Usage-based
MemOS	Structural Edges	Abstraction + Summarization	N/A	Agent-based	N/A
MemGAS	Structural Edges	N/A	N/A	N/A	N/A
LightMem	N/A	Summarization	N/A	Agent-based	Content-based

into a three-tier hierarchical structure: short-term memory for timely conversations, mid-term memory for topic summaries, and long-term memory for user preferences. By applying different, synergistic management and retrieval strategies to respective storage components, hierarchical storage effectively optimizes the trade-off between computational overhead and knowledge persistence.

③ Vector-based storage. This approach encodes textual memory into high-dimensional embeddings, subsequently indexed in dedicated vector libraries or databases, such as FAISS [14] and Qdrant, to enable the agent to perform efficient semantic similarity search. Vector-based storage can function as a standalone repository or serve as a foundational building block frequently integrated into more complex storage architectures.

④ Graph-based storage. This approach utilizes diverse graph topologies, such as trees, knowledge graphs, and temporal graphs, to preserve the rich structural information inherent in memory. For instance, MemTree organizes memory into a hierarchical tree where each node encapsulates aggregated textual content, providing varying levels of abstraction along the tree’s depth; Zep employs a layered temporal knowledge graph that concurrently organizes memory by representing raw messages as nodes, extracting subject–predicate–object triples, and clustering entities into communities. These graph-based storage methods capture intricate relationships and multi-hop associations that lie beyond the reach of simple vector similarity metrics.

7 INFORMATION RETRIEVAL

This module processes the incoming query q_t over the persistent memory state $\mathcal{H}_t$, identifying and ranking relevant memory records to produce the retrieved context $\mathcal{X}_t$ for downstream reasoning or context-aware response generation. Existing information retrieval strategies can be broadly categorized into four paradigms based on the fundamental mechanisms they employ:

① Lexical-Based Retrieval. This paradigm relies on the overlap of surface-level tokens or terms, typically implemented through some representative techniques such as set-based matching via the Jaccard similarity coefficient or scoring models like BM25 [90]. Lexical-based retrieval provides a strong baseline for exact term

matching, which can be particularly effective for retrieving names, specific entities, or phrases where precise wording is critical.

② Vector-Based Retrieval. This paradigm leverages semantic similarity in a continuous vector space to address the vocabulary mismatch problem inherent in exact keyword matching. By encoding both the query and memories into high-dimensional vectors via embedding models, vector-based retrieval is formulated as a top- k search for the most relevant entries using distance metrics like cosine similarity. This approach excels at capturing latent semantic nuances, ensuring that relevance is determined by semantic content rather than surface-level lexical form. To maintain efficiency within the massive scale of memory storage, Approximate Nearest Neighbor (ANN) search algorithms, such as HNSW [73] or PQ [38] (Product Quantization), are frequently employed.

③ Structure-Based Retrieval. This paradigm exploits the explicit relational connections between memory entities, often operating on graph-based or hierarchical storage, performing graph traversal, neighborhood expansion, or multi-hop reasoning to retrieve interconnected clusters of information instead of simple query-to-item matching. For example, Mem0^g explores the relationships starting from nodes identified through similarity search to construct a comprehensive subgraph that captures relevant and multi-faceted information. Similarly, Zep utilizes a BFS-based graph traversal algorithm to enhance initial search results by identifying additional nodes and edges.

④ LLM-Assisted Retrieval. This paradigm integrates LLMs as an active reasoning component to guide or refine the retrieval process. In addition to directly deciding which specific information should be retrieved, LLMs can also be utilized to transform ambiguous user prompts into precise search queries or to identify key entities within a query to facilitate more targeted retrieval. By leveraging the reasoning capabilities of LLMs, this paradigm excels at uncovering latent semantic dependencies, thereby ensuring closer alignment between queries and retrieved knowledge.

8 EXPERIMENTS

We now present the experimental results. We discuss the setup in Section 8.1, and then report the evaluation results across several experiments in Section 8.2.

8.1 Setup

► Workflow of our evaluation.

We conduct a systematic experimental study of agent memory mechanisms along three dimensions: (1) we collect and reimplement 12 representative methods within the unified framework described in Section 3; (2) we perform a comprehensive evaluation on two widely-used long-term conversational memory benchmarks and an agentic memory benchmark using multiple complementary metrics; (3) we carry out multi-dimensional analyses to assess architectural trade-offs and robustness, covering token cost and retrieval latency efficiency, ground-truth position sensitivity, context scalability, and LLM backbone dependence.

► **Benchmark Datasets.** We employ three benchmark datasets to evaluate the performance of each memory mechanism. To assess long-term conversational memory capabilities, we use LOCOMO [72] and LONGMEMEVAL [111], which represent two distinct interaction scenarios: human-human dialogues and user-AI interactions, respectively. We further use MemoryArena [29] to evaluate memory mechanisms in agent-environment interactions. Further dataset details are provided in Appendix B.1. Due to the configurable structure of LONGMEMEVAL, we construct specific variants to evaluate **context scalability** and **position sensitivity**. The detailed variant construction methodology is outlined in Appendix B.2.

► **Method Selection.** We focus on text-centric agent memory methods and select representative approaches based on their relevance, recency, reproducibility, and architectural diversity. As summarized in Table 1 and Table 2, the selected methods cover the major design choices across the four memory modules. Further method selection details are provided in Appendix F.

► **Evaluation Metrics.** For the conversational benchmarks, LOCOMO and LONGMEMEVAL, following their evaluation protocols and prior studies on long-horizon conversational memory, we adopt three complementary metrics. **F1** measures token-level overlap by balancing precision and recall. **BLEU-1** captures unigram-level modified precision with a brevity penalty, reflecting lexical fidelity to the reference answer. **Accuracy** is assessed by an LLM judge (GPT-5.4-mini) that determines whether the generated answer is semantically consistent with the reference answer. The specific prompt for LLM-as-a-judge is detailed in Appendix A.

For the agentic benchmark MemoryArena, we follow its official evaluation protocol and report **Task Success Rate (SR)**, which measures the proportion of fully completed tasks, and **Task Progress Score (PS)**, which measures partial task completion based on the proportion of satisfied task criteria. For *Progressive Web Search*, we additionally report **Accuracy**, measured by the correctness of the final search query in each task, and **#Search**, measured by the number of search-tool calls used to answer the final integrated query.

► **Implementation.** We implement all methods in Python under the proposed unified framework, ensuring faithful and consistent reimplementations based on original papers and publicly available code. All experiments are conducted on 8 NVIDIA A100 (80 GB) GPUs. If a method cannot finish in two days, we mark its result as “—” in the tables. Additionally, since F1 and BLEU-1 metrics are highly sensitive to answer verbosity, we apply a uniform simplification step

to all generated answers. The specific prompt and discussion for this process is detailed in Appendix A and Appendix E, respectively.

► **Hyperparameter Settings.** Unless otherwise stated, we use Qwen3.5-9B as the default LLM backbone, as it represents a capable and accessible open-source model. The maximum context length is set to 20,000 tokens, and we employ greedy decoding to ensure deterministic outputs. For all methods that involve top- k retrieval, following the previous work [43, 89, 115] and based on the sensitivity analysis in Section 8.2 (Exp.7), we set $k = 10$ to balance retrieval effectiveness and context length. We adopt all-MiniLM-L6-v2, a representative and widely used sentence-transformer model, as the unified embedding model across all methods. All remaining method-specific hyperparameters follow the original settings reported in their respective papers and codebases.

8.2 Evaluation

► **Exp.1. Overall performance.** We first report the performance of all agent memory methods on both benchmarks across two model scales (Qwen3.5-9B and Qwen3.5-27B). Results on LONGMEMEVAL and LOCOMO are shown in Table 3 and Table 4, respectively. Based on these results, we draw the following observations:

(1) Tree-based memory methods (e.g., MemTree and MemOS) generally achieve strong performance by organizing memory in a multi-layered, multi-granularity fashion. Specifically, MemTree achieves strong performance on LONGMEMEVAL, while MemOS demonstrates competitive results on LOCOMO. Tree structures provide high-level conceptual summaries at upper layers while preserving fine-grained details at leaf nodes. A similar advantage can be realized through well-designed hierarchical architectures that facilitate efficient information flow and transformation across different levels of abstraction, as evidenced by the highly competitive results of MemoryOS and Zep.

(2) Preserving information completeness is crucial for effective memory persistence—specifically, retaining raw messages during the information extraction phase and incorporating original conversations during final response generation. For example, methods that exclusively extract graph-based triples may suffer from information loss compared to those that preserve raw dialogue fragments, which may explain why Mem0 outperforms Mem0^g in many cases.

(3) Multi-session and temporal reasoning tasks remain highly sensitive to the reasoning capability of the backbone LLM. When scaling the backbone model from 9B to 27B, substantial performance improvements can be observed across different memory architectures. Specifically, on the Multi-Session category of LONGMEMEVAL, both MemTree and MemoryOS achieve more than 1.6× performance gains, while on the Temporal category of LOCOMO, Mem0 and MemOS improve their F1 scores by over 10 points. These results indicate that stronger LLM reasoning capabilities can significantly benefit memory-intensive tasks, while also motivating the development of memory architectures that provide explicit support for complex reasoning over persistent information.

► **Exp.2. Token cost and retrieval latency analysis.** In this experiment, we evaluate the efficiency of each method during both memory construction and retrieval. For memory construction, we report token consumption, rather than runtime, as a proxy for computational cost, since this stage is dominated by LLM calls and

Table 3: Comparison of methods on LONGMEMEVAL. The best and second-best results are marked in bold and underlined, respectively.

Method	Information Extraction						Multi-Session		Temporal		Knowledge Updates		Overall	
	user		assistant		preference									
	F1	BLEU-1	F1	BLEU-1	F1	BLEU-1	F1	BLEU-1	F1	BLEU-1	F1	BLEU-1	F1	BLEU-1
Qwen3.5-9B														
A-MEM	40.15	35.99	43.51	36.02	11.72	0.91	17.08	14.58	22.91	16.47	31.91	29.05	26.81	21.92
MemoryBank	56.17	46.14	66.64	55.02	20.67	9.54	17.70	10.73	22.41	14.75	37.20	27.84	33.04	24.32
MemGPT	55.54	49.79	54.67	46.00	13.63	3.82	19.19	17.19	22.30	15.86	29.54	27.07	30.36	25.37
Mem0	56.46	47.35	47.78	37.97	9.03	0.04	25.52	20.73	29.65	18.42	36.70	27.54	34.20	25.59
Mem0 ^g	55.56	48.44	22.93	18.25	11.85	0.47	<u>30.71</u>	28.41	35.45	<u>27.13</u>	40.86	33.42	35.03	28.84
MemoChat	12.37	6.01	31.00	23.37	10.69	0.91	21.85	17.94	31.36	21.03	11.25	8.41	21.75	15.19
Zep	—	—	—	—	—	—	—	—	—	—	—	—	—	—
MemTree	<u>68.18</u>	<u>63.67</u>	58.82	45.94	11.46	1.57	26.05	23.89	<u>36.61</u>	26.07	<u>49.69</u>	46.49	41.24	<u>34.69</u>
MemoryOS	65.23	59.56	<u>66.67</u>	<u>59.82</u>	12.87	1.66	27.29	24.16	36.38	29.76	43.06	39.31	<u>41.02</u>	35.61
MemOS	67.59	60.09	58.84	50.76	<u>14.28</u>	0.55	26.81	23.95	25.91	18.34	40.74	33.81	37.29	30.66
MemGAS	61.98	56.57	72.55	61.85	11.73	2.95	16.38	14.78	21.10	16.73	42.23	36.75	34.06	29.14
LightMem	70.12	65.05	18.33	13.97	13.13	<u>4.69</u>	31.67	<u>28.07</u>	39.33	27.05	49.76	<u>44.05</u>	39.31	32.49
Qwen3.5-27B														
A-MEM	57.37	49.83	51.76	42.14	11.87	0.20	22.33	20.05	28.31	23.01	43.84	30.90	34.85	27.98
MemoryBank	54.91	51.38	60.71	53.36	<u>13.62</u>	4.59	34.71	29.79	29.92	26.00	39.52	31.35	38.66	33.18
MemGPT	57.03	52.48	65.79	50.46	6.11	0.95	22.67	19.83	30.24	21.39	32.09	25.34	34.80	27.97
Mem0	70.20	62.93	50.05	41.67	13.95	0.99	33.03	31.60	36.27	29.16	48.23	44.17	42.23	36.59
Mem0 ^g	65.67	57.90	41.86	37.84	12.17	1.14	38.45	31.44	39.71	34.41	51.03	48.42	43.36	37.48
MemoChat	19.41	16.42	27.13	21.37	8.33	0.36	25.26	22.43	32.40	22.28	22.96	20.29	25.17	19.77
Zep	—	—	—	—	—	—	—	—	—	—	—	—	—	—
MemTree	70.29	62.22	72.22	61.83	11.41	1.03	44.75	39.86	<u>40.68</u>	<u>29.90</u>	53.06	48.96	<u>49.62</u>	<u>41.89</u>
MemoryOS	76.14	71.79	78.81	66.96	12.72	1.87	<u>44.44</u>	<u>38.32</u>	39.06	29.33	57.94	53.92	51.50	44.07
MemOS	72.14	66.07	56.48	45.35	13.38	0.55	35.55	30.49	37.14	22.72	53.80	44.27	44.96	35.42
MemGAS	67.98	61.79	<u>73.52</u>	<u>62.16</u>	12.74	4.21	17.26	16.51	22.70	18.69	46.57	41.29	36.41	31.67
LightMem	<u>72.52</u>	<u>66.25</u>	21.94	16.86	13.14	<u>4.43</u>	37.10	32.14	44.45	30.26	<u>57.06</u>	<u>50.18</u>	43.99	35.86

[†] The unavailable results for Zep are explained in Appendix H.

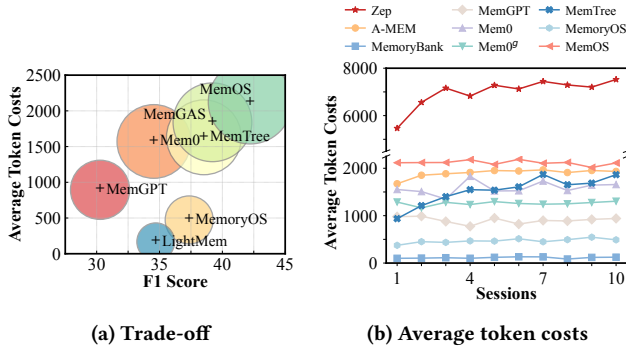


Figure 6: Performance and efficiency analysis on LOCOMO.

its runtime is highly sensitive to deployment environments and system conditions. Specifically, we analyze: (1) the overall trade-off between performance and token cost, where Figure 6a plots the average token cost per dialogue against the overall F1 score; and (2) construction scalability, where Figure 6b shows how the average token consumption evolves as the volume of ingested memory increases. For retrieval, Table 5 reports the average elapsed time from when a memory method receives a query until the retrieval process is completed, before response generation begins.

In general, higher performance correlates with increased token consumption, reflecting the benefit of extensive LLM utilization. While MemGAS and MemOS achieve high accuracy, they incur substantial token overhead. In contrast, MemoryOS and LightMem provide a better trade-off between performance and efficiency, achieving strong performance with significantly lower token costs. As memory volume grows, certain methods, such as MemTree and Zep, exhibit poor token-cost scalability due to increasing memory update overheads.

Retrieval latency varies substantially with the complexity of the retrieval pipeline. MemTree achieves the lowest retrieval latency (24.5ms), as it treats all tree nodes as a unified set and directly retrieves the relevant nodes without traversing the tree structure. LightMem, a representative lightweight memory method, achieves the second-lowest latency (78.1 ms). In contrast, methods involving query-time LLM calls, for example, to extract entities from the query, generally incur substantially higher retrieval latency.

► **Exp.3. Context scalability analysis.** In this experiment, we investigate the context scalability of various memory architectures by expanding the context length of LONGMEMEVAL from 50% to 200%. In long-horizon interactions, the primary challenge for memory systems shifts from simple retrieval to robust noise suppression

Table 4: Comparison of methods on LOCOMO. The best and second-best results are marked in bold and underlined, respectively.

Method	Single-Hop			Multi-Hop			Temporal			Open-Domain			Overall		
	F1	BLEU-1	Acc	F1	BLEU-1	Acc	F1	BLEU-1	Acc	F1	BLEU-1	Acc	F1	BLEU-1	Acc
Qwen3.5-9B															
A-MEM	34.38	30.22	53.75	23.26	15.18	52.13	18.30	13.12	52.02	12.38	8.78	39.58	27.62	22.57	52.21
MemoryBank	45.05	39.09	71.70	<u>36.93</u>	<u>28.77</u>	78.01	27.29	22.20	54.52	13.83	10.05	38.54	37.92	31.87	67.21
MemGPT	38.97	33.32	54.34	25.11	17.45	50.00	16.57	13.47	41.74	14.99	10.99	34.38	30.27	24.88	49.68
Mem0	34.66	29.75	63.26	29.69	21.36	68.79	<u>42.68</u>	<u>36.97</u>	61.68	20.66	14.70	45.83	34.55	28.78	62.86
Mem0 ^g	26.41	21.76	58.50	21.07	14.59	57.09	39.35	33.33	62.93	15.76	10.50	48.96	27.47	22.15	58.57
MemoChat	6.64	5.12	9.63	8.07	5.16	18.09	4.94	4.10	4.67	9.75	6.08	26.04	6.74	4.98	11.17
Zep	<u>48.72</u>	<u>42.98</u>	81.23	36.29	28.13	84.40	40.90	35.96	<u>71.19</u>	17.84	13.25	51.04	42.89	36.94	77.84
MemTree	43.42	36.24	69.23	34.21	22.57	71.92	33.08	27.60	<u>60.97</u>	26.54	21.28	58.30	38.53	31.01	72.85
MemoryOS	43.58	36.61	67.54	35.64	25.26	73.40	28.05	21.36	50.47	18.98	13.61	<u>56.25</u>	37.36	29.92	64.35
MemOS	45.38	40.16	74.53	38.41	30.60	<u>80.50</u>	43.45	39.24	72.26	<u>21.49</u>	14.97	52.08	<u>42.21</u>	<u>36.65</u>	<u>73.75</u>
MemGAS	51.55	45.26	<u>78.00</u>	36.28	28.43	76.60	15.33	11.42	36.76	19.47	<u>15.23</u>	46.88	39.21	33.25	67.21
LightMem	37.93	31.87	68.37	25.00	17.24	64.54	40.01	33.09	68.22	17.22	11.48	52.08	34.70	28.18	66.62
Qwen3.5-27B															
A-MEM	34.76	30.28	53.63	26.06	17.32	58.87	14.88	10.48	44.86	12.39	9.56	34.38	27.63	22.49	51.56
MemoryBank	46.56	40.24	75.51	40.38	<u>32.10</u>	78.72	34.38	28.49	61.37	18.69	14.59	43.75	41.15	34.70	71.17
MemGPT	50.98	44.54	72.53	32.08	22.89	62.41	20.85	16.95	49.53	18.29	14.53	40.62	39.20	32.95	63.90
Mem0	41.11	35.38	71.94	36.96	28.13	74.47	<u>52.97</u>	<u>46.33</u>	74.45	22.73	17.25	50.00	41.68	35.20	71.56
Mem0 ^g	38.48	33.16	68.85	39.30	29.71	78.37	52.69	46.16	74.14	<u>26.58</u>	19.76	57.29	40.85	34.40	70.97
MemoChat	6.93	5.46	8.80	6.52	3.66	14.18	7.08	6.05	4.98	10.98	7.22	22.92	7.14	5.36	9.87
Zep	53.15	46.29	84.37	37.92	29.41	86.17	45.19	39.66	74.45	19.68	15.14	55.21	<u>46.62</u>	<u>39.88</u>	80.81
MemTree	44.30	37.21	71.38	37.79	28.90	76.14	34.56	28.32	67.56	26.18	<u>19.97</u>	57.91	39.95	32.76	75.64
MemoryOS	50.71	44.27	76.34	<u>40.39</u>	30.09	79.43	37.22	29.27	62.31	24.20	18.16	61.46	44.36	36.92	73.05
MemOS	<u>52.04</u>	44.92	<u>82.33</u>	38.48	29.43	78.01	55.98	48.64	81.36	27.97	20.72	<u>60.42</u>	48.88	41.35	<u>79.97</u>
MemGAS	51.95	<u>45.73</u>	76.10	40.97	32.33	<u>82.62</u>	25.52	21.64	39.88	21.18	15.93	48.96	42.51	36.40	68.05
LightMem	40.69	34.23	72.06	27.62	19.56	67.02	52.82	44.32	<u>80.06</u>	22.22	16.73	56.25	39.67	32.55	71.82

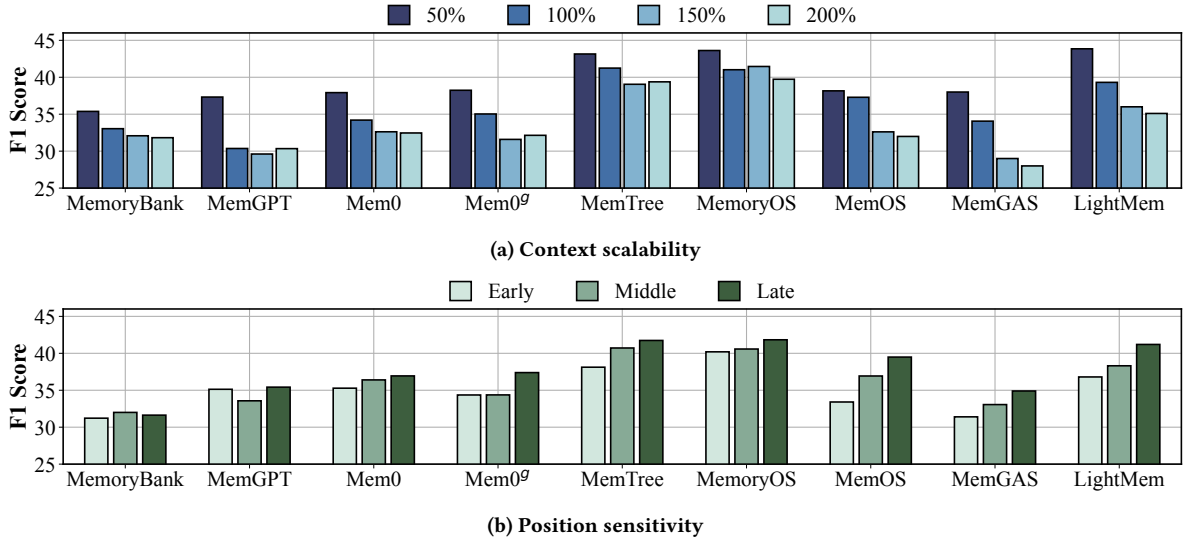


Figure 7: Robustness analysis of memory mechanisms on LONGMEMEVAL. (a) illustrates the context scalability as the input scale varies from 50% to 200%, while (b) shows the position sensitivity regarding ground-truth information placed at different relative positions: Early (first 1/3), Middle (middle 1/3), and Late (last 1/3).

as information density grows. We evaluate various architectures

across different context sizes to see how well they maintain retrieval precision as context grows. Figure 7a illustrates the overall

Table 5: Average retrieval latency per query (ms).

Method	Without Query-Time LLM Calls									With Query-Time LLM Calls		
	A-MEM	MemoryBank	MemGPT	Mem0	Zep	MemTree	MemOS	MemGAS	LightMem	Mem0 ^g	MemoChat	MemoryOS
Latency (ms)	617.1	864.8	105.8	356.4	536.8	24.5	346.7	793.4	78.1	3783.1	1792.4	2329.6

Table 6: Comparison across LLM backbones on LOCOMO.

Model	Metric	Zep	MemoryOS	MemOS	LightMem
Qwen3.5-9B	F1	42.89	37.36	42.21	34.70
	BLEU-1	36.94	29.92	36.65	28.18
Qwen3.5-27B	F1	46.62	44.36	48.88	39.67
	BLEU-1	39.88	36.92	41.35	32.55
DeepSeek-V4-Flash	F1	43.16	42.82	48.52	38.88
	BLEU-1	38.71	35.31	40.11	31.69
GPT-5.4-mini	F1	50.10	46.66	50.93	40.64
	BLEU-1	41.69	38.85	42.45	32.72

trends in context scalability. As the context scale expands from 50% to 200%, nearly all memory architectures exhibit a steady decline in F1 scores. This performance attrition is primarily driven by the increased density of irrelevant information, which lowers the signal-to-noise ratio during retrieval. Further analysis of this experiment is provided in Appendix B.

► **Exp.4. Position sensitivity analysis.** In this experiment, we evaluate how the placement of key evidence affects retrieval and reasoning on variants of LONGMEMEVAL by positioning the evidence in the early (first 1/3), middle (middle 1/3), or late (last 1/3) sections of the context. As evidence appears earlier in the context, memory systems must bridge larger temporal gaps and handle increased interference from subsequent dialogue. This experiment tests whether architectures maintain uniform access to historical records or exhibit a bias toward recent inputs. As illustrated in Figure 7b, a clear recency bias is observed at the overall level as the temporal distance between supporting evidence and the query increases. Further analysis of this experiment is provided in Appendix B.

► **Exp.5. LLM backbone comparison.** In this experiment, we evaluate several memory methods across different LLM backbones. Specifically, we select representative and high-performing methods from each design paradigm for this cross-backbone evaluation. We therefore exclude simpler baselines to focus on methods with non-trivial memory architectures. The results are presented in Table 6. Overall, most methods achieve their best performance with the closed-source GPT-5.4-mini backbone. Among open-source models, Qwen3.5-27B generally outperforms DeepSeek-V4-Flash across methods. Scaling Qwen3.5 from 9B to 27B produces substantial gains for all memory methods, as discussed in Exp.1, indicating that existing memory architectures remain heavily dependent on the backbone’s reasoning capability.

► **Exp.6. New SOTA algorithm.** Based on the above analysis, we design a new memory framework that achieves state-of-the-art performance among the evaluated representative memory baselines under the same experimental settings while maintaining low token overhead. Motivated by their strong performance, we integrate the

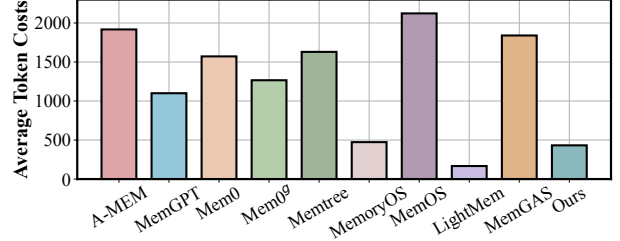


Figure 8: Comparison of our newly designed method in terms of average token costs.

Table 7: Comparison of our newly designed method on LOCOMO and LONGMEMEVAL.

Method	LOCOMO		LONGMEMEVAL	
	F1	BLEU-1	F1	BLEU-1
Zep	42.89	36.94	—	—
MemTree	38.53	31.01	41.24	34.69
MemoryOS	37.36	29.92	41.02	35.61
MemOS	42.21	36.65	37.29	30.66
LightMem	34.70	28.18	39.31	32.49
Ours	43.40	37.36	46.84	39.83

Table 8: Component ablations and organization variants measured by F1 score on LOCOMO.

Method Variants	SH	MH	TR	OK	Overall
Ours	49.63	36.94	39.86	19.69	43.40
w/o Tree	45.82	35.23	31.89	14.25	39.01
w/o SP	47.97	34.99	35.24	19.20	41.15
w/o HM & SP (MemTree)	43.42	34.21	33.08	26.54	38.53
w/o Tree & SP (MemoryOS)	43.58	35.64	28.05	18.98	37.36
w/o Tree, w/ Graph	45.52	33.68	40.06	19.59	40.60
w/o Tree & SP, w/ Graph	45.97	26.84	40.88	21.59	39.89

multi-granularity tree organization of MemTree and MemOS with the hierarchical memory storage of MemoryOS. To reduce the token overhead of tree updates, we further employ segment-level processing instead of processing individual dialogue turns. This design not only reduces update costs but also preserves semantic coherence, resulting in improved performance [83, 116]. The detailed workflow and algorithm are provided in Appendix C.

Table 7 reports the performance metrics, and Figure 8 compares the average token costs. Our method achieves the best overall performance on both benchmarks and delivers highly competitive

Table 9: Comparison of methods on MemoryArena. The best and second-best results are marked in bold and underlined, respectively.

Method	Bundled Web		Group Travel			Progressive Web		Formal Reasoning				All Task Avg SR
	Shopping		Planning			Search		Math		Phys		
	SR	PS	SR	PS	sPS	Acc	#Search	SR	PS	SR	PS	
A-MEM	0.00	31.03	0.00	<u>0.27</u>	6.32	2.26	7.04	10.00	<u>25.92</u>	35.00	56.54	9.45
MemoryBank	0.00	32.08	0.00	0.21	5.97	5.43	6.09	<u>17.50</u>	24.49	35.00	<u>54.04</u>	11.59
MemGPT/Letta	0.00	36.45	0.00	0.00	4.82	3.62	7.09	20.00	28.58	20.00	49.04	8.72
Mem $\emptyset$	0.00	31.91	0.00	0.00	<u>7.48</u>	5.43	7.78	10.00	23.49	30.00	49.18	9.09
Mem $\emptyset^g$	0.00	<u>32.50</u>	0.00	0.00	7.34	<u>4.98</u>	7.60	12.50	24.47	10.00	46.97	5.50
Zep	0.00	27.56	0.00	0.05	6.60	2.26	4.49	15.00	23.61	45.00	52.42	12.45
MemTree	0.00	28.89	0.00	0.16	6.29	2.26	<u>4.96</u>	12.50	24.54	30.00	46.22	8.95
MemoryOS	0.00	30.22	0.00	0.43	8.02	2.26	8.13	15.00	21.80	15.00	42.51	6.45
MemOS	0.00	26.44	0.00	0.05	4.09	1.81	8.33	15.00	24.78	15.00	32.88	6.36
MemGAS	0.00	18.67	0.00	0.00	5.90	3.62	6.25	15.00	21.31	<u>40.00</u>	45.92	<u>11.72</u>
LightMem	0.00	30.11	0.00	0.00	0.08	1.36	10.51	7.50	20.93	25.00	35.26	6.77

results across all task categories, while maintaining a remarkably low computational overhead of fewer than 450 tokens per dialogue.

We further conduct component-wise ablations on three key designs: tree-based organization (Tree), hierarchical memory separation (HM), and segment-level processing (SP), as reported in Table 8. Note that SH, MH, TR, and OK denote single-hop, multi-hop, temporal reasoning, and open-domain knowledge questions, respectively. Since SP operates during the transition from short-term to mid-term memory, removing HM necessarily removes SP. Removing Tree or SP reduces the overall F1 score by 4.39 and 2.25 points, respectively, while retaining only Tree (MemTree) or HM (MemoryOS) results in larger performance drops. These results demonstrate the contribution of all three components, with temporal reasoning being the most affected question category.

We also replace Tree with Graph while keeping the remaining framework unchanged, following Zep’s knowledge-graph construction pipeline. Replacing Tree with Graph reduces the overall F1 score from 43.40 to 40.60, showing that tree-based organization provides stronger and more balanced overall performance. Nevertheless, Graph performs better on temporal reasoning questions, suggesting that the temporal information retained in its nodes and edges is more effective for modeling temporal dependencies.

► **Exp.7. Sensitivity to retrieval top- k .** We evaluate representative methods using different retrieval top- k values, with $k \in \{1, 3, 5, 10, 15\}$, on LOCOMO, as shown in Figure 9. Performance improves substantially as k increases from 1 to 10, while the gains become marginal beyond $k=10$. Based on this observation and following the default settings adopted in prior work [43, 89, 115], we use $k=10$ as the default unless otherwise specified.

► **Exp.8. Evaluation on Agentic Memory Tasks.** In this experiment, we evaluate representative memory methods on MemoryArena, which measures memory-augmented agents in agent-environment interactions. The results are shown in Table 9. Overall, existing approaches still face substantial challenges in achieving reliable task completion. Most methods achieve low SR scores, especially on *Bundled Web Shopping* and *Group Travel Planning*, where an SR of zero indicates that the agent fails to complete any full task trajectory. Zep achieves the best overall performance with

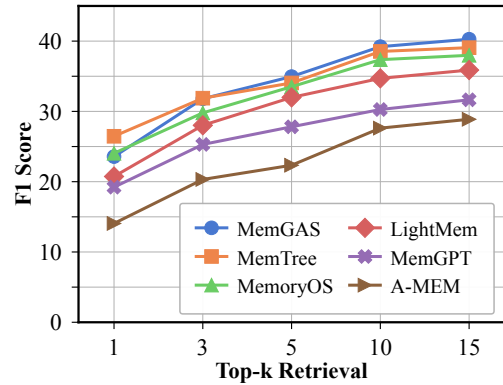


Figure 9: F1 performance of different memory methods under varying retrieval top- k values.

an average SR of 12.45, followed by MemGAS and MemoryBank. The low success rates across tasks indicate that existing memory mechanisms are still insufficient for supporting long-horizon agentic interactions.

9 LESSONS AND OPPORTUNITIES

We summarize the lessons (L) for practitioners based on our observations and propose practical research opportunities (O).

Lessons:

- **L1.** Based on the findings from all conducted experiments, we develop a roadmap in Figure 10 for selecting suitable memory architectures across four core capabilities: information persistence, memory association, temporal reasoning, and knowledge update. The detailed supporting analyses are provided in Appendix G.
- **L2.** Compared to flat memory structures, hierarchical organization is more effective in capturing structural relationships between information, which can be achieved by either employing tree-based indices or designing multi-level storage.
- **L3.** Information completeness is fundamental to memory mechanisms. While structured representations like triples in graphs improve organization, retaining raw dialogue context is essential to

Table 10: Positioning of our work against representative benchmarks and surveys on agent memory.

Work	Primary Type	Design Taxonomy	Unified Multi-method Evaluation	Component-Performance Linkage	Evidence-guided Design Synthesis
<i>Memory in the Age of AI Agents</i> [34]	Survey	✓	✗	✗	✗
<i>The AI Hippocampus: How Far Are We From Human Memory?</i> [40]	Survey	✓	✗	✗	✗
<i>A Survey on the Memory Mechanism of Large Language Model Based Agents</i> [125]	Survey	✓	✗	✗	✗
<i>Rethinking Memory Mechanisms of Foundation Agents in the Second Half</i> [37]	Survey	✓	✗	✗	✗
<i>LOCOMO</i> [72]	Benchmark	✗	✗	✓	✗
<i>LongMemEval</i> [111]	Benchmark	✓	✗	✓	✓
<i>MemoryAgentBench</i> [35]	Benchmark	✗	✓	✗	✗
<i>MemoryArena</i> [29]	Benchmark	✗	✓	✗	✗
Memory in the LLM Era	Empirical Study	✓	✓	✓	✓

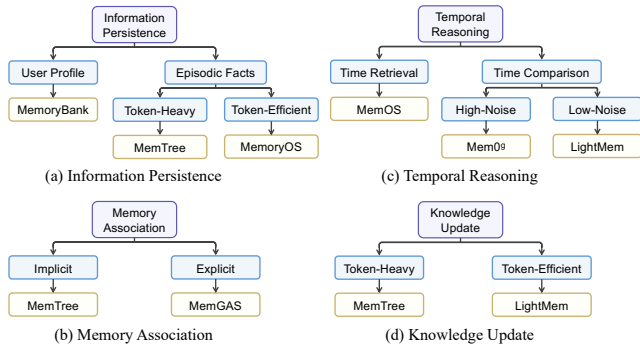


Figure 10: Task-Oriented Roadmap for Selecting Long-Term Memory Architectures under Different Scenarios.

prevent semantic loss during the information extraction or retrieval stage.

Opportunities:

► **Q1.** In real-world settings, memory involves complex and diverse information sources, including textual conversations, historical interaction traces, and multimodal signals such as audio, images, and videos. Existing memory mechanisms often focus on a single or limited form, which constrains effective information utilization. A promising future research direction is to develop unified memory mechanisms that support heterogeneous and multimodal memory within a shared storage and retrieval framework.

► **Q2.** Existing competitive memory methods often manage and maintain information in ways that lead to rapid growth in storage size and increasing management and retrieval overhead. How to compress memory without losing useful information remains a major challenge, which creates an opportunity to explore latent representations beyond explicit text and learned compression mechanisms to achieve high-density yet usable memory.

10 RELATED WORKS

In this section, we briefly review the prior work most relevant to our study, including RAG frameworks and LLMs for database.

• **RAG frameworks.** RAG has been proven to excel in many tasks, including open-ended question answering [39, 97], programming context [8–10], SQL rewrite [59, 102], automatic DBMS configuration debugging [96, 129], and data cleaning [74, 75, 87]. The

naive RAG technique relies on retrieving query-relevant information from external knowledge bases to mitigate the “hallucination” of LLMs. Recently, most RAG approaches [16, 26, 27, 50, 86, 91, 108, 112] have adopted graphs as the external knowledge to organize the information and relationships within documents, achieving improved overall retrieval performance. In terms of open-source software, a variety of graph databases are supported by both the LangChain [46] and LlamaIndex [69] libraries, while a more general class of graph-based RAG applications is also emerging, including systems that can create and reason over knowledge graphs in both Neo4j [77] and NebulaGraph [76]. For more details, please refer to the recent surveys and experimental studies of graph-based RAG methods [28, 86, 131].

• **LLMs for database.** Due to the wealth of developer experience captured in a vast array of database forum discussions, recent studies [8, 17, 23, 47, 56, 59, 98, 102, 129, 130] have begun leveraging LLMs to enhance database performance. For instance, GPTuner [47] proposes to enhance database knob tuning using LLMs by leveraging domain knowledge to identify important knobs and coarsely initialize their values for subsequent refinement. Besides, D-Bot [129] proposes an LLM-based database diagnosis system, which can retrieve relevant knowledge chunks and tools, and use them to identify typical root causes accurately. The LLM-based data analysis systems and tools have also been studied [2, 11, 20, 30, 52, 53, 60–64, 85, 118, 123].

• **Agent Memory and Data Management.** Agent memory provides the essential data management foundation for stateful LLM agents. It continuously ingests interaction streams, transforms them into logical memory records, maintains evolving and potentially conflicting information, organizes these records in textual, vector, hierarchical, or graph forms, and retrieves relevant context for incoming queries. Together, these operations constitute a data management pipeline for persistent agent states. This data management perspective is also reflected in recent database research [7, 19, 44, 101]. Fernandez et al. [19] highlight the complementary role of databases and information retrieval systems in supplying and processing external information for LLMs. GaussDB-Vector [101] explicitly treats a persistent vector database as long-term memory for LLM applications, supporting scalable search

and real-time updates. Khan et al. [44] also identify data organization, indexing, retrieval, and updates in RAG as data management challenges.

• **Agent Memory Surveys and Benchmarks.** Recent surveys provide conceptual taxonomies of agent memory from different perspectives [15, 34, 37, 40, 41, 71, 113, 114, 120, 125], while recent memory benchmarks (e.g., LOCOMO [72], LONGMEMEVAL [111], MemoryAgentBench [35], and MemoryArena [29]) establish evaluation protocols for conversational and agentic memory capabilities. As summarized in Table 10, these studies primarily focus on conceptual organization or system-level evaluation. Our work complements them by decomposing representative memory methods into reusable memory modules, evaluating these methods under unified settings, and empirically connecting component-level design choices to performance differences. These findings further support evidence-guided memory design and architecture selection.

11 CONCLUSIONS

In this paper, we provide an in-depth experimental evaluation and comparison of existing memory methods. We first present a unified modular framework that abstracts memory mechanisms into four core memory modules—information extraction, memory management, memory storage, and information retrieval. Under this framework, we systematically evaluate representative memory methods on three benchmark datasets, and further conduct multi-dimensional experiments to study token cost efficiency, context scalability, evidence position sensitivity and LLM backbone dependence. Based on experimental results and analyses, we develop a new memory variant by combining existing techniques, which achieves strong accuracy while maintaining low overhead. Finally, we summarize the lessons learned and propose practical research opportunities that can facilitate future studies.

REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmerschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A Shah, et al. 2024. The Design of an LLM-powered Unstructured Analytics System. *arXiv preprint arXiv:2409.00847* (2024).
- [3] Anthropic. 2026. Introducing Claude Sonnet 4.6: Our fastest, smartest model is now available for all. <https://www.anthropic.com/news/claude-sonnet-4-6>.
- [4] Akari Asai, Zequi Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection. *arXiv preprint arXiv:2310.11511* (2023).
- [5] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 3119–3137. <https://doi.org/10.18653/v1/2024.acl-long.172>
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bf8ac142f64a-Paper.pdf
- [7] Edward Y Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4874–4886.
- [8] Sibe Chen, Ju Fan, Bin Wu, Nan Tang, Chao Deng, Pengyi Wang, Ye Li, Jian Tan, Feifei Li, Jingren Zhou, et al. 2024. Automatic Database Configuration Debugging using Retrieval-Augmented Language Models. *arXiv preprint arXiv:2412.07548* (2024).
- [9] Sibe Chen, Yeye He, Weiwei Cui, Ju Fan, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. 2024. Auto-Formula: Recommend Formulas in Spreadsheets using Contrastive Learning for Table Representations. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [10] Sibe Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. Haipipe: Combining Human-Generated and Machine-Generated Pipelines for Data Preparation. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [11] Zui Chen, Lei Cao, Sam Madden, Tim Kraska, Zeyuan Shang, Ju Fan, Nan Tang, Zihui Gu, Chunwei Liu, and Michael Cafarella. 2023. SEED: Domain-Specific Data Curation With Large Language Models. *arXiv e-prints* (2023), arXiv–2310.
- [12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. <https://arxiv.org/abs/2504.19413> arXiv:2504.19413.
- [13] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Tianyu Liu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. 2024. A Survey on In-Context Learning. *arXiv preprint arXiv:2301.00234* (2024). <https://arxiv.org/abs/2301.00234>
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The Faiss Library. arXiv:2401.08281 [cs.LG] <https://arxiv.org/abs/2401.08281>
- [15] Yiming Du, Wenyu Huang, Danna Zheng, Zhaowei Wang, Sebastian Montella, Mirella Lapata, Kam-Fai Wong, and Jeff Z. Pan. 2025. Rethinking Memory in LLM based Agents: Representations, Operations, and Emerging Topics. arXiv:2505.00675 [cs.CL] <https://arxiv.org/abs/2505.00675>
- [16] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [17] Ju Fan, Zihui Gu, Songyue Zhang, Yuxin Zhang, Zui Chen, Lei Cao, Guoliang Li, Samuel Madden, Xiaoyong Du, and Nan Tang. 2024. Combining small language models and large language models for zero-shot n2sql. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2750–2763.
- [18] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. 2025. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866* (2025).
- [19] Raul Castro Fernandez, Aaron J Elmore, Michael J Franklin, Sanjay Krishnan, and Chenhao Tan. 2023. How large language models will disrupt data management. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3302–3309.
- [20] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (Jan. 2024), 1132–1145. <https://doi.org/10.14778/3641204.3641221>
- [21] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).
- [22] Aashish Ghimire, James Prather, and John Edwards. 2024. Generative AI in Education: A Study of Educators’ Awareness, Sentiments, and Influencing Factors. *arXiv preprint arXiv:2403.15586* (2024).
- [23] Victor Giannankouris and Immanuel Trummer. 2024. $\{\lambda\}$ -Tune: Harnessing Large Language Models for Automated Database System Tuning. *arXiv preprint arXiv:2411.03500* (2024).
- [24] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. 2016. Hybrid Computing Using a Neural Network with Dynamic External Memory. *Nature* 538, 7626 (2016), 471–476. <https://doi.org/10.1038/nature20101>
- [25] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *The Innovation* (2024).
- [26] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv e-prints* (2024), arXiv–2410.
- [27] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=hkjujAPVsg>
- [28] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-Augmented Generation with Graphs (GraphRAG). *arXiv preprint arXiv:2501.00309* (2024).
- [29] Zexue He, Yu Wang, Churan Zhi, Yanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. 2026. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. *arXiv preprint arXiv:2602.16313* (2026).
- [30] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Jiang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Robert Tang, Xiangtao Lu, Xiaowu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Yongxin Ni, Zhibin Gou, Zongze Xu, Yuyu Luo, and Chenglin Wu. 2025. Data Interpreter: An LLM Agent for Data Science. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 19796–19821. <https://doi.org/10.18653/v1/2025.findings-acl.1016>
- [31] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekeshe, Fei Jia, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models?. In *Proceedings of the First Conference on Language Modeling (COLM)*.
- [32] Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. ChatDB: Augmenting LLMs with Databases as Their Symbolic Memory. arXiv:2306.03901 [cs.AI] <https://arxiv.org/abs/2306.03901>
- [33] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. HiAgent: Hierarchical Working Memory Management for Solving Long-Horizon Agent Tasks with Large Language Model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 32779–32798. <https://doi.org/10.18653/v1/2025.acl-long.1575>
- [34] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564* (2025).
- [35] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257* (2025).
- [36] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv preprint arXiv:2311.05232* (2023).
- [37] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, et al. 2026. Rethinking Memory Mechanisms of Foundation Agents in the Second Half: A Survey. *arXiv preprint arXiv:2602.06052* (2026).

- [38] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [39] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403* (2024).
- [40] Zixia Jia, Jiaqi Li, Yipeng Kang, Yuxuan Wang, Tong Wu, Quansen Wang, Xiaobo Wang, Shuyi Zhang, Junzhe Shen, Qing Li, et al. 2026. The AI Hippocampus: How Far are We From Human Memory? *arXiv preprint arXiv:2601.09113* (2026).
- [41] Dongming Jiang, Yi Li, Songtao Wei, Jinxin Yang, Ayushi Kishore, Alysa Zhao, Dingyi Kang, Xu Hu, Feng Chen, Qiannan Li, et al. 2026. Anatomy of agentic memory: Taxonomy and empirical analysis of evaluation and system limitations. *arXiv preprint arXiv:2602.19320* (2026).
- [42] Tomoyuki Kagaya, Thong Jing Yuan, Yuxuan Lou, Jayashree Karlekar, Sugiri Pranata, Akira Kinose, Koki Oguri, Felix Wick, and Yang You. 2024. Rap: Retrieval-augmented planning with contextual memory for multimodal llm agents. *arXiv preprint arXiv:2402.03610* (2024).
- [43] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. *arXiv preprint arXiv:2506.06326* (2025).
- [44] Arijit Khan, Yuyu Luo, Wenjie Zhang, Mingqi Zhou, and Xiaofang Zhou. 2025. Retrieval-augmented Generation (RAG): What is There for Data Management Researchers? *SIGMOD record* 54, 4 (2025), 33.
- [45] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714* (2023).
- [46] Langchain. 2023. Langchain. https://python.langchain.com/docs/additional_resources/arxiv_references/.
- [47] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanguo Chen, Mingjie Tang, and Jianguo Wang. 2024. Gptuner: A manual-reading database tuning system via gpt-guided bayesian optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952.
- [48] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.
- [49] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. 2025. From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2757–2791.
- [50] Dawei Li, Shu Yang, Zhen Tan, Jae Young Baik, Sukwon Yun, Joseph Lee, Aaron Chacko, Bojian Hou, Duy Duong-Tran, Ying Ding, et al. 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to answer Alzheimer’s Disease Questions with Scientific Literature. *arXiv preprint arXiv:2405.04819* (2024).
- [51] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llm-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [52] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and Zhao-Xiang Zhang. 2023. SheetCopilot: Bringing Software Productivity to the Next Level Through Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. 4952–4984.
- [53] Lan Li, Liri Fang, Bertram Ludäscher, and Vette I Torvik. 2025. AutoDCWorkflow: LLM-based Data Cleaning Workflow Auto-Generation and Benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 7766–7780. <https://doi.org/10.18653/v1/2025.findings-emnlp.410>
- [54] Xinzhe Li. 2024. A Review of Prominent Paradigms for LLM-Based Agents: Tool Use (Including RAG), Planning, and Feedback Learning. *arXiv:2406.05804* [cs.AI] <https://arxiv.org/abs/2406.05804>
- [55] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houa Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 6342–6353. <https://doi.org/10.18653/v1/2023.emnlp-main.391>
- [56] Yiyang Li, Haoyang Li, Jing Zhang, Renata Borovica-Gajic, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. 2025. AgentTune: An Agent-Based Large Language Model Framework for Database Knob Tuning. *Proc. ACM Manag. Data* 3, 6, Article 293 (Dec. 2025), 29 pages. <https://doi.org/10.1145/3769758>
- [57] Yinheng Li, Shaofei Wang, Han Ding, and Hang Chen. 2023. Large Language Models in Finance: A Survey. In *Proceedings of the fourth ACM international conference on AI in finance*. 374–382.
- [58] Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Chen, Jiawei Yang, Chunyu Li, Qingchen Yu, et al. 2025. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724* (2025).
- [59] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2025. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proceedings of the VLDB Endowment* 1, 18 (2025), 53–65.
- [60] Chen Liang, Donghua Yang, Zheng Liang, Zhiyu Liang, Tianle Zhang, Boyu Xiao, Yuqing Yang, Wenqi Wang, and Hongzhi Wang. 2025. Revisiting Data Analysis with Pre-trained Foundation Models. *arXiv preprint arXiv:2501.01631* (2025).
- [61] Yiming Lin, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G Parameswaran. 2025. TWIX: Automatically Reconstructing Structured Data from Templated Documents. *arXiv preprint arXiv:2501.06659* (2025).
- [62] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *arXiv preprint arXiv:2405.04674* (2024).
- [63] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. *arXiv preprint arXiv:2405.14696* (2024).
- [64] Chunwei Liu, Gerardo Vitagliano, Brandon Rose, Matt Prinz, David Andrew Samson, and Michael Cafarella. 2025. PalimpChat: Declarative and Interactive AI Analytics. *arXiv preprint arXiv:2502.03368* (2025).
- [65] Lei Liu, Xiaoyan Yang, Junchi Lei, Xiaoyang Liu, Yue Shen, Zhiqiang Zhang, Peng Wei, Jinjie Gu, Zhixuan Chu, Zhan Qin, et al. 2024. A Survey on Medical Large Language Models: Technology, Application, Trustworthiness, and Future Directions. *arXiv preprint arXiv:2406.03712* (2024).
- [66] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.
- [67] Peiqi Liu, Zhanqiu Guo, Mohit Warke, Soumith Chintala, Chris Paxton, Nur Muhammad Mahi Shafullah, and Llerel Pinto. 2025. DynaMem: Online Dynamic Spatio-Semantic Memory for Open World Mobile Manipulation. In *ICRA 2025 Workshop: Human-Centered Robot Learning in the Era of Big Data and Large Models*. <https://openreview.net/forum?id=RJKUHDJg1>
- [68] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *arXiv:2107.13586* [cs.CL] <https://arxiv.org/abs/2107.13586>
- [69] llamaindex. 2023. llamaindex. <https://www.llamaindex.ai/>.
- [70] Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. 2023. MemoChat: Tuning LLMs to Use Memos for Consistent Long-Range Open-Domain Conversation. *arXiv preprint arXiv:2308.08239* (2023).
- [71] Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. 2026. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms. *arXiv:2605.06716* [cs.AI] <https://arxiv.org/abs/2605.06716>
- [72] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. <https://arxiv.org/abs/2402.17753> *arXiv:2402.17753*.
- [73] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [74] Zan Ahmad Naeem, Mohammad Shahmeer Ahmad, Mohamed Eltabakh, Mourad Ouzzani, and Nan Tang. 2024. RetClean: Retrieval-Based Data Cleaning Using LLMs and Data Lakes. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4421–4424.
- [75] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proceedings of the VLDB Endowment* 16, 4 (2022), 738–746.
- [76] NebulaGraph. 2024. NebulaGraph. <https://nebula-graph.io/>.
- [77] Neo4j. 2006. Neo4j. <https://neo4j.com/>.
- [78] Yiqi Nie, Yaxuan Kong, Xiaowen Dong, John M Mulvey, H Vincent Poor, Qingsong Wen, and Stefan Zohren. 2024. A Survey of Large Language Models for Financial Applications: Progress, Prospects and Challenges. *arXiv preprint arXiv:2406.11903* (2024).
- [79] OpenClaw Team. 2026. OpenClaw: Your own personal AI assistant. <https://github.com/openclaw/openclaw>.
- [80] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, Vol. 35. 27730–27744.

- [81] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. 2025. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140* (2025).
- [82] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).
- [83] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H Vicky Zhao, Lili Qiu, et al. 2025. On memory construction and retrieval for personalized conversational agents. *arXiv preprint arXiv:2502.05589* (2025).
- [84] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. Association for Computing Machinery. <https://doi.org/10.1145/3586183.3606763>
- [85] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling semantic queries with llms over tables of unstructured and structured data. *arXiv preprint arXiv:2407.11418* (2024).
- [86] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. *arXiv preprint arXiv:2408.08921* (2024).
- [87] Yichen Qian, Yongyi He, Rong Zhu, Jintao Huang, Zhijian Ma, Haibin Wang, Yaohua Wang, Xiuyu Sun, Defu Lian, Bolin Ding, et al. 2024. UniDM: A Unified Framework for Data Manipulation with Large Language Models. *Proceedings of Machine Learning and Systems* 6 (2024), 465–482.
- [88] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).
- [89] Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2024. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. *arXiv preprint arXiv:2410.14052* (2024).
- [90] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (April 2009), 333–389. <https://doi.org/10.1561/15000000019>
- [91] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *arXiv preprint arXiv:2401.18059* (2024).
- [92] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2024. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [93] Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. 2023. ZeroSCROLLS: A Zero-Shot Benchmark for Long Text Understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 7977–7989. <https://doi.org/10.18653/v1/2023.findings-emnlp.536>
- [94] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 377, 8634–8652 pages.
- [95] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. OpenAI GPT-5 System Card. *arXiv:2601.03267 [cs.CL]* <https://arxiv.org/abs/2601.03267>
- [96] Vikramank Singh, Kapil Eknath Vaidya, Vinayashakar Bannihatti Kumar, Sopan Khosla, Murali Narayanaswamy, Rashmi Gangadharaiah, and Tim Kraska. 2024. Panda: Performance Debugging for Databases Using LLM Agents. (2024).
- [97] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. 2023. Improving the Domain Adaptation of Retrieval Augmented Generation (RAG) Models for Open Domain Question Answering. *Transactions of the Association for Computational Linguistics* 11 (2023), 1–17.
- [98] Tarek Stolz, István Koren, Liam Tirpitz, and Sandra Geisler. 2023. GALOIS: A Hybrid and Platform-Agnostic Stream Processing Architecture. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments* (Seattle, WA, USA) (BiDEDE '23). Association for Computing Machinery, New York, NY, USA, Article 3, 6 pages. <https://doi.org/10.1145/3579142.3594287>
- [99] Theodore Sumers, Shunyu Yao, Karthik R Narasimhan, and Thomas L. Griffiths. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=1i6ZCvflQJ> Survey Certification, Featured Certification.
- [100] Haoran Sun, Shaoning Zeng, and Bob Zhang. 2026. H-MEM: Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vera Demberg, Kentaro Inui, and Lluis Marquez (Eds.). Association for Computational Linguistics, Rabat, Morocco, 341–350. <https://doi.org/10.18653/v1/2026.eacl-long.15>
- [101] Ji Sun, Guoliang Li, James Pan, Jiang Wang, Yongqing Xie, Ruicheng Liu, and Wen Nie. 2025. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4951–4963.
- [102] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2024. R-Bot: An LLM-based Query Rewrite System. *arXiv preprint arXiv:2412.01661* (2024).
- [103] Ajay Guyyala Prudhvi Ratna Badri Satya Vijay Putta, Krishna Teja Areti. 2026. Self-Reflective Memory Consolidation in Agentic Architectures. *International Journal of Computer Applications* 187, 73 (Jan 2026), 1–14. <https://doi.org/10.5120/ijca2026926236>
- [104] Jinqiang Wang, Huansheng Ning, Yi Peng, Qikai Wei, Daniel Tesfai, Wenwei Mao, Tao Zhu, and Runhe Huang. 2024. A Survey on Large Language Models from General Purpose to Medical Applications: Datasets, Methodologies, and Evaluations. *arXiv preprint arXiv:2406.10303* (2024).
- [105] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- [106] Shen Wang, Tianlong Xu, Hang Li, Chaoli Zhang, Joleen Liang, Jiliang Tang, Philip S Yu, and Qingsong Wen. 2024. Large Language Models for Education: A Survey and Outlook. *arXiv preprint arXiv:2403.18105* (2024).
- [107] Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. 2025. M+: Extending MemoryLLM with scalable long-term memory. *arXiv preprint arXiv:2502.00592* (2025).
- [108] Yu Wang, Nedom Lipka, Ryan A Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19206–19214.
- [109] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent workflow memory. *arXiv preprint arXiv:2409.07429* (2024).
- [110] Lilian Weng. 2023. LLM Powered Autonomous Agents. lilianweng.github.io. <https://lilianweng.github.io/posts/2023-06-23-agent/>
- [111] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=pZiyCaVuti>
- [112] Junde Wu, Jiayuan Zhu, Yunli Qi, Jingkun Chen, Min Xu, Filippo Menolascina, and Vicente Grau. 2024. Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2408.04187* (2024).
- [113] Shanglin Wu and Kai Shu. 2026. Memory in llm-based multi-agent systems: Mechanisms, challenges, and collective intelligence. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 167–184.
- [114] Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. 2025. From human memory to ai memory: A survey on memory mechanisms in the era of llms. *arXiv preprint arXiv:2504.15965* (2025).
- [115] Xu Wujiang, Mei Kai, Gao Hang, Tan Juntao, Liang Zujie, and Zhang Yongfeng. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025).
- [116] Derong Xu, Yi Wen, Pengyue Jia, Yingyi Zhang, Wenlin Zhang, Yichao Wang, Huifeng Guo, Ruiming Tang, Xiangyu Zhao, Enhong Chen, and Tong Xu. 2026. From Single to Multi-Granularity: Toward Long-Term Memory Association and Selection of Conversational Agents. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=i2ylvZARnG>
- [117] Jing Xu, Arthur Szlam, and Jason Weston. 2022. Beyond Goldfish Memory: Long-Term Open-Domain Conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 5180–5197. <https://doi.org/10.18653/v1/2022.acl-long.356>
- [118] Zihan Yan, Rui Xi, and Mengshu Hou. 2025. MCTuner: Spatial Decomposition-Enhanced Database Tuning via LLM-Guided Exploration. *Proc. ACM Manag. Data* 3, 6, Article 342 (Dec. 2025), 25 pages. <https://doi.org/10.1145/3769807>
- [119] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [120] Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, Yujing Zhang, Zhu Wang, Zijin Hong, Zheng Yuan, Zhishang Xiang, Shengyuan Chen, Huachi Zhou, Qinggang Zhang, Ninghao Liu, Jinsong Su, Xinrun Wang, Yi Chang, and Xiao Huang. 2026. Graph-based Agent Memory: Taxonomy, Techniques, and Applications. *arXiv:2602.05665 [cs.AI]* <https://arxiv.org/abs/2602.05665>
- [121] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793 [cs.SE]*

- <https://arxiv.org/abs/2405.15793>
- [122] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629 [cs.CL]* <https://arxiv.org/abs/2210.03629>
 - [123] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2024. Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8754–8782. <https://doi.org/10.18653/v1/2024.emnlp-main.497>
 - [124] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. 2018. Personalizing Dialogue Agents: I Have a Dog, Do You Have Pets Too? *arXiv:1801.07243 [cs.AI]* <https://arxiv.org/abs/1801.07243>
 - [125] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.
 - [126] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36. 46595–46623.
 - [127] Yanxin Zheng, Wensheng Gan, Zefeng Chen, Zhenlian Qi, Qian Liang, and Philip S Yu. 2024. Large Language Models for Medicine: A Survey. *International Journal of Machine Learning and Cybernetics* (2024), 1–26.
 - [128] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731.
 - [129] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-bot: Database diagnosis system using large language models. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2514–2527.
 - [130] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. Db-gpt: Large language model meets database. *Data Science and Engineering* 9, 1 (2024), 102–111.
 - [131] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, and Yixiang Fang. 2026. In-Depth Analysis of Graph-Based RAG in a Unified Framework. *Proceedings of the VLDB Endowment* 18, 13 (Jan. 2026), 5623–5637. <https://doi.org/10.14778/3773731.3773738>
 - [132] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841* (2025).

Prompt for Graph-based Extraction.

Entity Extraction:

Previous messages:
{previous messages}
Current message:
{current message}
Given the above conversation, extract entity nodes from the current message that are explicitly or implicitly mentioned. Note that ALWAYS extract the speaker/actor as the first node and extract other significant entities, concepts, or actors mentioned in the current message.

INSTRUCTIONS:

1. ALWAYS extract the speaker/actor as the first node. The speaker is the part before the colon in each line of dialogue.
2. Extract other significant entities, concepts, or actors mentioned in the current message.
3. DO NOT create nodes for relationships or actions.
4. DO NOT create nodes for temporal information like dates, times or years (these will be added to edges later).
5. Be as explicit as possible in your node names, using full names.
6. DO NOT extract entities mentioned only in the previous messages.

Relation Extraction:

Previous messages:
{previous messages}
Current message:
{current message}
Entities:
{entities}
Given the above messages and entities, extract all facts pertaining to the listed entities from the current message. Note that extract facts only between the provided entities and each fact should represent a clear relationship between two DISTINCT nodes.

INSTRUCTIONS:

1. Extract facts only between the provided entities.
2. Each fact should represent a clear relationship between two DISTINCT nodes.
3. The relation_type should be a concise, all-caps description of the fact (e.g., LOVES, IS_FRIENDS_WITH, WORKS_FOR).
4. Provide a more detailed fact containing all relevant information.
5. Consider temporal aspects of relationships when relevant.

Figure 11: A sample prompt for graph-based extraction.

A PROMPT TEMPLATES

This section details several prompt templates used in our evaluation. A sample prompt for graph-based information extraction is

shown in Figure 11, the prompt for LLM-as-a-judge is detailed in Figure 14, and the prompt for post-processing answer simplification is provided in Figure 12.

B EXPERIMENT DETAILS

B.1 Dataset Details

In this section, we provide a detailed overview of the three benchmarks.

The LOCOMO benchmark [72] comprises ten long-term conversations for question-answering evaluation. Each conversation features an average of 198.6 questions spanning 27.2 sessions and approximately 588.2 dialogue turns between two speakers. The specific task categories are defined as follows:

- **Single-hop** questions require answers based on a single session.
- **Multi-hop** questions require synthesizing information from multiple different sessions.
- **Temporal reasoning** questions require temporal reasoning and capturing time-related data cues.
- **Open-domain knowledge** questions require integrating provided information with external knowledge such as common-sense or world facts.

The LONGMEMEVAL benchmark [111] contains 500 high-quality questions designed to evaluate four core long-term memory abilities. Each question is grounded in a dedicated conversation history based on long-term user-AI interactions, averaging 50.2 sessions and approximately 115,000 tokens in length. LONGMEMEVAL categorizes memory tasks to assess the following aspects:

- **Information Extraction (IE):** Ability to recall specific information from extensive interactive histories, including the details mentioned by either the user (**single-session-user**) or the assistant (**single-session-assistant**), and whether the model can utilize the user information to generate a personalized response (**single-session-preference**).
- **Multi-Session Reasoning (MR):** Ability to synthesize the information across multiple history sessions to answer complex questions that involve aggregation and comparison.
- **Knowledge Updates (KU):** Ability to recognize the changes in the user’s personal information and update the knowledge of the user dynamically over time.
- **Temporal Reasoning (TR):** Awareness of the temporal aspects of user information, including both explicit time mentions and timestamp metadata in the interactions.

The MemoryArena benchmark [29] comprises multi-session agentic tasks designed to evaluate memory use in agent-environment interactions. Each task contains an average of 6.9 explicitly interdependent subtasks and approximately 57 agent actions. Tasks are categorized into four types:

- **Bundled Web Shopping:** The agent sequentially purchases a bundle of related products in a web environment. Each later purchase must be compatible with previously selected items while satisfying additional constraints such as budget, price, or rating, thereby testing whether the agent can retain and reuse earlier decisions.
- **Group Travel Planning:** Starting from a finalized itinerary for one traveler, additional travelers join incrementally with

Prompt for Answer Simplification.

Your task is to act as an answer simplifier. I will give you a question and a full-sentence answer. You must reduce the answer to its most critical component.

Follow these rules:

1. ****Extract the Core Information:**** Identify the primary piece of information that directly answers the question.
2. ****Remove Extraneous Phrases:**** Eliminate phrases like "Based on the information provided...", "The answer is...", "As per the document...", etc.
3. ****Omit Explanations:**** Do not include any justifications, reasoning, or additional context from the original answer.
4. ****Be Concise:**** The output should be the shortest possible string that still accurately answers the question.

Example:

- Question: "What degree did I graduate with?"
- Original Answer: "Based on the information provided, you graduated with a degree in Business Administration."
- Simplified Answer: "Business Administration"

Here is the question and answer to simplify:

- Question: {question}
- Original Answer: {answer}
- Simplified Answer:

Figure 12: Prompt for answer simplification.

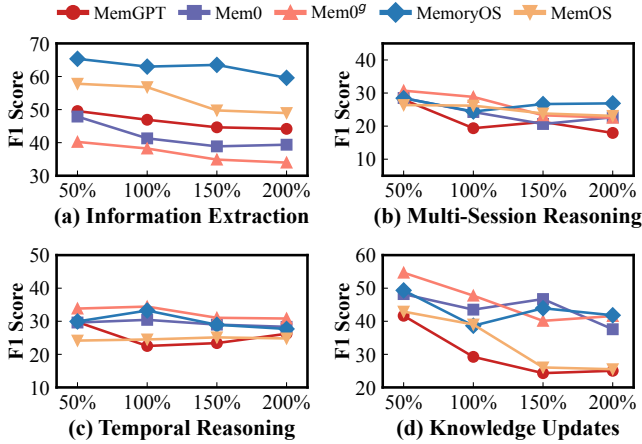


Figure 13: Context scalability of various memory methods across different task categories on LONGMEMEVAL.

shared or personalized preferences. The agent must generate each new itinerary by referring to previous travelers' plans and resolving cross-traveler constraints, such as joining an existing activity or selecting an option relative to another traveler's choice.

- *Progressive Web Search*: A complex search problem is decomposed into a sequence of dependent subqueries, with each

subquery introducing an additional condition. The agent must preserve and integrate results from earlier searches so that the final answer satisfies all previously introduced constraints.

- *Sequential Formal Reasoning*: Research-level mathematics and physics problems are decomposed into ordered chains of intermediate questions. Later questions depend on definitions, lemmas, and intermediate results established in earlier sessions, requiring the agent to retrieve and reuse prior reasoning correctly.

It therefore evaluates several complementary aspects of agent memory, including cross-session state retention, tool-assisted interaction, long-horizon constraint tracking, and the reuse of previously acquired information or reasoning results.

For implementation, we followed the official MemoryArena source code and evaluation workflow, while using Qwen3.5-9B as the LLM backbone. We mainly report **Task Success Rate (SR)** and **Task Progress Score (PS)**. Given N tasks, where task S_i contains $|S_i|$ ordered subtasks and $|S_i^{\text{pass}}|$ of them are successfully completed, the metrics are defined as:

$$\text{SR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(S_i \text{ is successfully completed}),$$

and

$$\text{PS} = \frac{1}{N} \sum_{i=1}^N \frac{|S_i^{\text{pass}}|}{|S_i|}.$$

SR measures end-to-end task completion, while PS provides a finer-grained measure of partial progress across interdependent subtasks. For *Group Travel Planning*, we additionally report the soft Progress Score (sPS), where each subtask receives partial credit based on the fraction of constraints it satisfies. The subtask scores are first averaged within each task and then averaged across all tasks. For *Progressive Web Search*, we report Accuracy, measured by the correctness of the final search query in each task, and #Search, measured by the number of search-tool calls used to answer the final integrated query.

B.2 Dataset Variant Construction

LONGMEMEVAL is well-suited for controlled evaluation: it features a configurable context length enabled by a *session pool*, which supplies topically similar history sessions without introducing conflicting information, and each sample is associated with a single question, making the corresponding ground-truth evidence clearly defined. Based on these properties, we construct a set of variants of the original dataset (LONGMEMEVAL_S) to evaluate **context scalability** and **position sensitivity**.

For **context scalability**, we construct variants at 50%, 150%, and 200% of the default context length. Specifically, for the 50% variant, we prune half of the history dialogues at session-level granularity while retaining the ground-truth session; for the 150% and 200% variants, we append additional sessions sampled from the session pool to the end of the original conversation history.

For **position sensitivity**, we generate position-varying variants of LONGMEMEVAL_S by relocating the ground-truth session. Specifically, we extract the ground-truth session from its original position and re-insert it into one of three equally partitioned segments of

Prompt for LLM-as-a-judge.

Your task is to label an answer to a question as “CORRECT” or “WRONG”. You will be given the following data: (1) a question (posed by one user to another user), (2) a ‘gold’ (ground truth) answer, (3) a generated answer which you will score as CORRECT/WRONG.

The point of the question is to ask about something one user should know about the other user based on their prior conversations. The gold answer will usually be a concise and short answer that includes the referenced topic, for example:

Question: Do you remember what I got the last time I went to Hawaii?

Gold answer: A shell necklace

The generated answer might be much longer, but you should be generous with your grading — as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.

For time-related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references (like ‘last Tuesday’ or ‘next month’), but you should be generous with your grading - as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs (e.g., ‘May 7th’ vs ‘7 May’), consider it CORRECT if it’s the same date.

Now it’s time for the real question:

Question: {question}

Gold answer: {gold_answer}

Generated answer: {generated_answer}

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG. Do NOT include both CORRECT and WRONG in your response, or it will break the evaluation script.

Just return the label CORRECT or WRONG in a json format with the key as “label”.

Figure 14: Prompt for LLM-as-a-judge.

the conversation: the first 1/3 (*Early*), the middle 1/3 (*Middle*), and the final 1/3 (*Late*).

B.3 Extended Context Scalability Analysis

This section provides an extended analysis of context scalability under increasing context lengths.

The scaling results reveal a performance divide rooted in operational complexity. Methods such as MemOS and MemGPT implement an “LLM-as-OS” paradigm, requiring the LLM to autonomously manage memory via complex tool calls. As the scale increases to 200%, the expanded candidate space significantly raises the difficulty

for the LLM to accurately reason over and execute these management instructions, leading to higher rates of tool-call failures and indexing conflicts. In contrast, MemoryOS reduces the agent’s cognitive load by employing explicit rule-based hierarchical management. By offloading organizational logic from the LLM to a deterministic framework, MemoryOS maintains high stability, demonstrating that simplifying the agent’s internal management overhead through deliberate design is critical for robust scaling.

Different task categories exhibit varying sensitivity to scaling pressure. As illustrated in Figure 13, Knowledge Update (KU) is particularly sensitive, showing sharp performance attrition because an increased memory volume raises the density of conflicting records. Since KU requires the model to identify the latest fact among mutually exclusive versions, the presence of more obsolete candidates directly increases retrieval interference. Conversely, temporal tasks remain relatively stable, as they rely on the relative ordering of events, which remains structurally distinct even as the background volume grows. Unlike the versioning conflicts in KU, the chronological precedence of events is not easily compromised by the addition of context.

Table 11: Position sensitivity analysis of representative memory mechanisms across Information Extraction sub-tasks on LONGMEMEVAL (F1 Score).

Method	Position	user	assistant	preference
MemOS	Early	63.80	23.35	11.39
	Middle	57.60	20.39	12.87
	Late	74.60	20.74	12.90
	Improvement	+10.80	-2.61	+1.51
MemTree	Early	59.01	55.42	10.42
	Middle	66.91	67.61	12.07
	Late	71.49	65.41	11.33
	Improvement	+12.48	+9.99	+0.91
MemOS	Early	69.40	49.09	12.64
	Middle	73.81	56.78	13.72
	Late	79.85	58.66	13.12
	Improvement	+10.45	+9.56	+0.48

B.4 Extended Position Sensitivity Analysis

This section provides an extended analysis of position sensitivity with key evidence placed at different positions.

A clear recency bias is observed at the overall level as the temporal distance between supporting evidence and the query increases. As shown in Figure 7b, most methods achieve higher overall F1 scores when the evidence is placed in late sessions rather than early sessions. As more intervening dialogue accumulates after the relevant evidence appears, maintaining long-range information consistency and retrieval becomes increasingly challenging.

Memory update policy significantly affects position sensitivity. For methods like A-MEM, dynamic memory revisions can overwrite earlier evidence, making early-session information vulnerable to later interactions. Hierarchical methods like MemTree and MemOS exhibit different mechanisms: although raw dialogue is retained at leaf nodes, updates to higher-level summaries amplify recent information’s influence, increasing the Late–Early gap. In contrast,

MemoryOS preserves historical information more evenly through stage-wise transfers across memory levels. Earlier evidence remains relatively independent within each level rather than being repeatedly merged with later information. Consequently, later interactions do not directly reshape earlier evidence representations, explaining MemoryOS’s smaller Late–Early gap.

Position sensitivity is highly category-dependent. We examine three representative methods in Table 11 across information extraction sub-tasks. Transient, session-localized information exhibits stronger position sensitivity than persistent traits. The user and assistant extraction tasks show substantial Early-to-Late changes while preference extraction remains stable. This indicates that position sensitivity correlates with information persistence: transient session-local details suffer more from later interference, while persistent preference traits are less affected by evidence relocation.

Algorithm 2: Our Newly Designed Method

```

input : Current query  $q$ , Short-term memory  $M_S$ , Mid-term
        memory  $M_M$ , Long-term memory tree  $M_L$ , short-term
        capacity threshold  $\tau$ , heat threshold  $\theta$ , beam width  $k$ 
output: Updated memories ( $M'_S, M'_M, M'_L$ ), Response  $r$ 
// (1) Information Retrieval
1  $C_S \leftarrow M_S$ ; // retrieve entire short-term memory
2  $C_{M\_flat} \leftarrow \text{VectorSearch}(M_M.\text{high-level-nodes}, q)$ ;
3  $C_{M\_beam} \leftarrow \text{BeamSearch}(M_M.\text{tree}, q, k)$ ;
4  $C_L \leftarrow \text{VectorSearch}(M_L, q)$ ;
5  $C \leftarrow C_S \cup C_{M\_flat} \cup C_{M\_beam} \cup C_L$ ; // aggregate retrieved
   context
// (2) Response Generation
6  $r \leftarrow \text{LLM\_Generate}(q, C)$ ;
// (3) Memory Ingestion
7  $M'_S \leftarrow \text{Enqueue}(M_S, q, r)$ ; // ingest new messages via
   short-term FIFO queue
8 if  $|M'_S| > \tau$  then
9    $M_{old} \leftarrow \text{DequeueOldestHalf}(M'_S)$ ;
10   $S_{new} \leftarrow \text{SegmentBySemanticSimilarity}(M_{old})$ ;
   // partition into segments
11   $M'_M \leftarrow \text{UpdateMemoryTree}(M_M, S_{new})$ ; // leaf:
   segment, parents: aggregated summary
12 else
13    $M'_M \leftarrow M_M$ ;
14   $M'_L \leftarrow M_L$ ;
15 foreach leaf node  $n \in M'_M.\text{leaves}$  do
16    $score \leftarrow \text{ComputeHeatScore}(n.\text{freq}, n.\text{recency})$ ;
17   if  $score > \theta$  then
18      $M'_L \leftarrow M'_L \cup \{n\}$ ; // promote high-heat segments
19 return ( $M'_S, M'_M, M'_L, r$ )

```

C DETAILS OF OUR NEWLY DESIGNED METHOD

This section details the complete workflow of our newly designed method and the algorithm is provided in Algorithm 2.

As illustrated in Figure 15, new messages are first ingested into short-term memory and managed via a FIFO queue. When the short-term memory reaches capacity, the oldest messages are partitioned

into segments based on semantic similarity and transferred to mid-term memory. Within this tier, we maintain a memory tree in which each leaf node represents a segment—capturing a generated summary of its constituent messages—while parent nodes provide aggregated summaries of their respective children. This segment-level granularity significantly reduces token overhead compared to turn-level processing. For each segment leaf node, we compute a *heat score* based on access frequency and recency; segments with high heat scores are promoted to long-term memory.

During the information retrieval stage, we conduct independent retrieval from each of the three storage tiers. Short-term memory is retrieved in its entirety to maintain context continuity. For mid-term memory, we employ a dual-mode retrieval mechanism: high-level node semantics are matched via flat vector-based similarity search, while raw messages are retrieved through a beam search that traverses from the root node, selecting the top- k most similar nodes at each level and ultimately reaching the raw messages stored as children of the segment leaf nodes. Long-term memory is accessed via standard vector-based similarity retrieval.

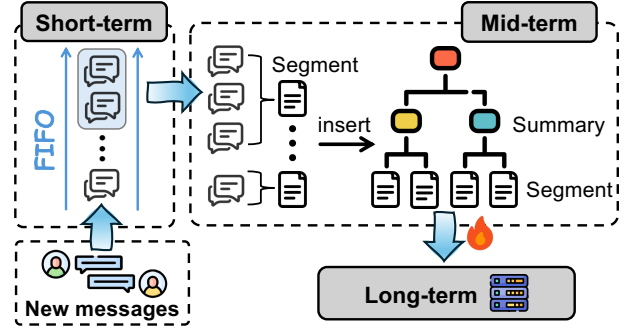


Figure 15: The framework of our newly designed method.

D MORE LESSONS AND OPPORTUNITIES

Lessons:

► **L4.** Optimizing memory granularity by processing multiple dialogue turns as a single unit during the information extraction or memory management stage significantly reduces token consumption, while appropriate partitioning further maintains the coherence of retrieved information.

► **L5.** Well-designed memory frameworks can reduce the dependence of reasoning performance on model scale, enabling smaller LLMs to effectively handle complex queries such as those involving temporal dependencies. For instance, employing multi-step reasoning or specialized components within the framework allows smaller LLMs to address complex tasks more effectively.

► **L6.** Instead of destructive updates, memory systems should adopt non-destructive strategies that preserve historical information while annotating its validity, enabling future reuse and preventing the loss of potentially useful knowledge. In other words, instead of deleting old information, systems should keep it and mark its status.

Opportunities:

► **O3.** Existing hierarchical memory mechanisms primarily focus on consolidating short-term memory into long-term storage but do not support the reverse transformation. A promising direction is to design bidirectional memory transformation mechanisms that

Table 12: Example of the effect of answer simplification on F1 and BLEU-1 scores.

Question:	When did Caroline go to the LGBTQ support group?		
Answer:	7 May 2023		
Response type	Predicted answer	F1	BLEU-1
Correct (before simplification)	Caroline went to the LGBTQ support group on May 7, 2023.	0.3750	0.2307
Correct (after simplification)	May 7, 2023	0.8571	0.7500
Incorrect	8 May 2023	0.6667	0.6667

Dialogue: I know Pluto is officially classified as a dwarf planet, but in the astronomy course I teach, we still categorize Pluto as a planet.
Q. How does the user categorize Pluto in their course?
A. Planet.

Predicted Answer:
The user categorizes Pluto as a **planet**.

LLM judge:
Incorrect, because Pluto is officially a dwarf planet.

Figure 16: An example where a correctly generated answer is incorrectly evaluated by an LLM judge.

enable efficient consolidation and reconstruction across memory hierarchies.

► **O4.** In complex query scenarios, relying on a single fixed retrieval strategy can be brittle and may lead to task failure. Different queries often require different retrieval granularity and mechanisms. However, existing memory systems do not consider this issue, so designing a retrieval routing planner that dynamically selects and adapts retrieval strategies to diverse memory-query contexts is an interesting research problem.

► **O5.** Existing memory benchmarks, such as LOCOMO and LONG-MEMOVAL, evaluate memory methods on pre-collected, static interaction histories, which do not reflect the continuous and evolving nature of real-world memory. They also fail to capture key properties of human memory such as the preference for recent updates and the strengthening of repeatedly mentioned information. Moreover, despite the inclusion of sparse visual elements in LOCOMO, these benchmarks remain strictly text-centric. Developing more challenging, interaction-driven, and comprehensive multimodal benchmarks that better reflect realistic memory formation and usage scenarios is a meaningful research direction.

E ANALYSIS OF EVALUATION METRICS

E.1 LLM-as-a-judge

It is worth noting that LLM-as-a-judge evaluation is not perfectly reliable, as its results may be affected by the capability of the judge model, the evaluation prompt, and other factors [25, 49, 51, 126]. Figure 16 presents a concrete example of this issue. Therefore, we use it as a complementary metric rather than a primary evaluation metric.

E.2 F1 and BLEU-1

Token-level F1 measures the token overlap between a generated answer Y and its reference answer Y^* . Specifically, precision, recall, and F1 are calculated as

$$P = \frac{|Y \cap Y^*|}{|Y|}, R = \frac{|Y \cap Y^*|}{|Y^*|}, F1 = \frac{2PR}{P + R}, \quad (1)$$

where $|Y \cap Y^*|$ denotes the number of overlapping tokens, with repeated tokens counted according to their minimum occurrence count in the two answers. BLEU-1 measures clipped unigram precision with a brevity penalty and is calculated as

$$\text{BLEU-1} = \text{BP} \cdot p_1, \quad (2)$$

where

$$p_1 = \frac{\sum_w \min(\text{Count}_Y(w), \text{Count}_{Y^*}(w))}{\sum_w \text{Count}_Y(w)}, \quad (3)$$

and the brevity penalty is defined as

$$\text{BP} = \begin{cases} 1, & |Y| \geq |Y^*|, \\ \exp\left(1 - \frac{|Y^*|}{|Y|}\right), & |Y| < |Y^*|. \end{cases} \quad (4)$$

Here, w denotes a unigram token, and $\text{Count}_Y(w)$ and $\text{Count}_{Y^*}(w)$ denote the number of occurrences of w in the generated and reference answers, respectively.

Therefore, F1 and BLEU-1 are sensitive not only to whether the generated answer contains the correct information, but also to its wording and verbosity. Additional explanatory content may introduce unmatched tokens and reduce lexical precision even when the key answer is correct. This issue is particularly relevant because the official implementations of different memory methods use different response prompts, resulting in differences in answer length and style. To reduce this prompt-induced output-style bias and more accurately compare the information contained in the answers, we apply the same simplification procedure to the responses of all methods before computing F1 and BLEU-1. Table 12 presents a concrete example illustrating how simplification affects both metrics, where removing semantically redundant details substantially increases the scores of the correct response and prevents it from being ranked below an incorrect but lexically closer answer. Table 13 reports the F1 and BLEU-1 scores of different methods before and after simplification, thereby directly quantifying its impact on the evaluation results.

Table 13: F1 and BLEU-1 scores of different methods before and after answer simplification on LOCOMO.

Method	F1			BLEU-1		
	Before	After	Δ	Before	After	Δ
A-MEM	26.90	27.62	+0.72	21.98	22.57	+0.59
Mem0	31.40	34.55	+3.15	25.32	28.78	+3.80
Zep	35.93	42.89	+6.96	30.78	36.94	+6.16
MemTree	35.36	38.53	+3.17	29.88	31.01	+1.13
MemoryOS	35.72	37.36	+1.64	26.15	29.92	+3.77
LightMem	29.88	34.70	+4.82	24.79	28.18	+3.39

Table 14: Classification of the selected memory baselines by storage organization and memory representation.

Organization	Vector	Graph	Tree
Flat	MemoryBank	Mem0 ^g	MemTree
Hierarchical	MemoryOS	Zep	MemOS

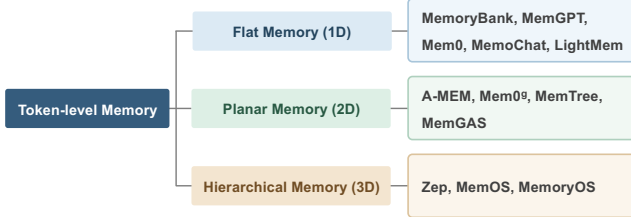


Figure 17: Classification and coverage of the evaluated token-level memory baselines following the taxonomy in *Memory in the Age of AI Agents: A Survey*.

F DISCUSSION OF METHOD SELECTION

In this section, we explain the coverage of the selected baselines from two complementary perspectives: the taxonomy in *Memory in the Age of AI Agents: A Survey* [34] and our four-stage framework.

Following *Memory in the Age of AI Agents: A Survey*, which categorizes agent memory into token-level, parametric, and latent memory, we focus on text-based token-level memory. Unlike parametric and latent memory, these methods can be evaluated under the same frozen-LLM setting without additional training or model modification. Within token-level memory, the survey further categorizes existing methods into Flat Memory (1D), Planar Memory (2D), and Hierarchical Memory (3D). As shown in Figure 17, for Flat Memory, we select representative methods covering classic conversational memory (e.g., MemoChat and MemoryBank), widely adopted designs (e.g., Mem0), efficiency-oriented designs (e.g., LightMem), and operating-system-inspired memory management (e.g., MemGPT). For Planar Memory, we include associative memory (e.g., A-MEM), knowledge-graph-based memory (e.g., Mem0g), tree-structured memory (e.g., MemTree), and multi-granularity graph memory (e.g., MemGAS). For Hierarchical Memory, we select representative multi-level designs, including MemoryOS, MemOS, and the layered temporal graph architecture Zep.

Beyond this survey-based taxonomy, the selected methods also cover diverse design choices across all four stages of our framework, as summarized in Table 1. Specifically, for *information extraction*, the selected methods cover direct archiving at both the message level (e.g., MemoryBank) and segment level (e.g., MemoryOS), summarization-based extraction (e.g., A-MEM), graph-based extraction (e.g., Zep and Mem0g), as well as the multi-granularity extraction design, MemGAS. For *memory management*, we include representative agentic approaches (e.g., Mem0 and LightMem) and operating-system-inspired approaches (e.g., MemGPT and MemOS). For *memory storage*, as shown in Table 14, the selected methods cover representative combinations across both organization-centric and representation-centric dimensions. For *information retrieval*, they cover lexical-based, vector-based, and structure-based retrieval, as well as hybrid designs that combine multiple retrieval mechanisms, such as Zep and MemGAS.

G SUPPORTING ANALYSIS FOR THE MEMORY ARCHITECTURE SELECTION ROADMAP

In this section, we provide the supporting analysis for the memory architecture selection roadmap. The roadmap covers four major capabilities of long-term memory systems: *Information Persistence*, *Memory Association*, *Temporal Reasoning*, and *Knowledge Update*. These capabilities are derived from the major QA categories in widely used benchmarks such as LOCOMO and LONGMEMEVAL. They offer complementary perspectives on long-term memory requirements and collectively provide broad coverage of common QA scenarios. The method recommendations are primarily based on the comprehensive performance of different methods across LOCOMO and LONGMEMEVAL. When multiple methods achieve comparable effectiveness but are better suited to different scenarios, we provide scenario-specific recommendations by further considering factors such as token consumption and robustness under different noise levels.

For *Information Persistence*, user-profile scenarios require memory mechanisms that model and maintain user preferences, as represented by MemoryBank. For episodic facts involving retrieval of specific information from historical interactions, MemTree is suitable when higher token overhead is acceptable, whereas the hierarchical design of MemoryOS provides a more token-efficient solution.

For *Memory Association*, the roadmap distinguishes implicit from explicit associations. Implicit associations, exemplified by LONGMEMEVAL’s Multi-Session Reasoning tasks, require integrating semantically related evidence distributed across sessions and levels of abstraction; MemTree supports this setting through its multi-granular tree organization. Explicit associations, corresponding to LOCOMO’s Multi-Hop questions, require multi-hop reasoning over structured paths connecting multiple memory units, which is supported by MemGAS through its multi-granularity graph organization.

For *Temporal Reasoning*, the roadmap distinguishes two representative types of questions. Time retrieval, mainly represented by the temporal questions in LOCOMO, focuses on retrieving information associated with a specific temporal reference. For this type of questions, MemOS achieves the strongest performance. Time comparison, largely represented by LONGMEMEVAL’s temporal reasoning tasks, requires comparing temporal relations among multiple pieces of information. As discussed in Section 8.2 of our revised paper (Exp.3. Context Scalability Analysis), increasing context length introduces higher noise levels. Therefore, we further distinguish low-noise and high-noise settings. Under low-noise histories, LightMem performs best, while under high-noise histories, Mem0^g achieves more robust performance.

For *Knowledge Update*, the principal trade-off is between update effectiveness and token efficiency. MemTree provides better update performance when substantial update cost is acceptable, whereas the lightweight update design of LightMem is more appropriate under tight token budgets.

H MEMORY CONSTRUCTION COST OF ZEP

The missing score indicates that Zep did not complete memory construction on LONGMEMEVAL within two days in our unified experimental environment. The “fast” performance reported in

Zep’s documentation¹ refers to retrieval latency, which is separately evaluated in Table 5, rather than index construction time. In our experiments, Zep exhibits an average retrieval latency of 536.8 ms, which is higher than the sub-200 ms latency reported in its documentation. This difference arises because, to ensure a unified and controlled experimental setting, we use the open-source implementation released for the Zep paper rather than its production API, which has undergone multiple subsequent iterations and optimizations. Regarding index construction, Zep requires several hours to build the index for LOCOMO under our experimental setting. For LONGMEMEVAL, whose aggregate text volume is over 100× that of LOCOMO, index construction could not be completed within two days.

To clarify this issue, we further analyze the sources of index construction overhead. Specifically, Zep represents each message as an episode and incrementally updates a temporal knowledge graph through Graphiti². For each episode, it performs entity and fact extraction, entity resolution, fact deduplication, and conflict handling. As the conversation grows, the expanding graph increases the candidate space for these operations, while longer context requires more information to be processed in LLM prompts. Consequently, later episodes generally incur higher processing costs, and repeatedly applying this pipeline over LONGMEMEVAL leads to substantial cumulative construction overhead.

¹<https://www.getzep.com/>

²<https://github.com/getzep/graphiti>